



Windowed Encoding and Phase-Aware Deep Reinforcement Learning for Deterministic Time-Dependent TSP

Author:

Anuradha DISSANAYAKE
1747776

Supervisors:

Prof. Dr. Dr. Lars
SCHMIDT-THIEME
Tim DERNEDDE

15th May 2026

Thesis submitted for
MASTER OF SCIENCE IN DATA ANALYTICS

WIRTSCHAFTSINFORMATIK UND MASCHINELLES LERNEN
STIFTUNG UNIVERSITÄT HILDESHEIM
UNIVERSITÄTSPLÄTZ 1, 31141 HILDESHEIM

Erklärung über das selbstständige Verfassen von:

Windowed Encoding and Phase-Aware Deep Reinforcement Learning for Deterministic Time-Dependent TSP.

Ich versichere hiermit, dass ich die vorstehende Masterarbeit selbstständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der obigen Arbeit, die anderen Werken dem Wortlaut oder dem Sinn nach entnommen wurden, habe ich in jedem einzelnen Fall durch die Angabe der Quelle bzw. der Herkunft, auch der benutzten Sekundärliteratur, als Entlehnung kenntlich gemacht. Dies gilt auch für Zeichnungen, Skizzen, bildliche Darstellungen sowie für Quellen aus dem Internet und anderen elektronischen Text- und Datensammlungen und dergleichen. Die eingereichte Arbeit ist nicht anderweitig als Prüfungsleistung verwendet worden oder in englischer oder in einer anderen Sprache als Veröffentlichung erschienen. Mir ist bewusst, dass wahrheitswidrige Angaben als Täuschung behandelt werden. Ich versichere weiterhin, dass ggf. direkt an die Prüfenden zur Begutachtung übermittelte gedruckte Exemplare der Masterarbeit mit der im Prüfungsamt eingereichten elektronischen Fassung der Arbeit übereinstimmen.

Statement as to the sole authorship of the thesis:

Windowed Encoding and Phase-Aware Deep Reinforcement Learning for Deterministic Time-Dependent TSP.

I hereby certify that the master's thesis named above was solely written by me and that no assistance was used other than that cited. The passages in this thesis that were taken verbatim or with the same sense as that of other works have been identified in each individual case by the citation of the source or the origin, including the secondary sources used. This also applies for drawings, sketches, illustration as well as internet sources and other collections of electronic texts or data, etc. The submitted thesis has not been previously used for the fulfilment of degree requirements and has not been published in English or any other language. I am aware of the fact that false declarations will be treated as fraud.

Date, City, Signature



19.05.2026 Hildesheim

Abstract

The Time-Dependent Travelling Salesman Problem (TDTSP) is an extended version of the classical TSP, in which travel times vary with the departure time instead of being based only on distance. This makes the TDTSP a strong match for real-world routing scenarios. The baseline paper (Zhang et al., 2023) introduced the M1 model using a deep reinforcement learning (DRL) approach, demonstrating strong performance on static routing problems. The M1 model is used as the baseline for this thesis, and three major gaps are identified and addressed through three architectural extensions, namely Decoder MLP variants, Cost-Aware Gating, and Time Slicing with a safe refresh mechanism. All experiments are evaluated on a real-world Beijing traffic dataset across customer set sizes of 19 to 49 nodes. Among all three architectural modifications, the Temperature MLP, which scales the attention logits using an MLP output, shows the best performance with up to a 4.96% cost reduction at $C = 49$. Cost-Aware Gating blends heuristic logits with the learned attention signal, failing to improve performance at $C = 19$ but achieving a 3.67% gain at $C = 49$. Time slicing with the safe refresh mechanism replaces the full-day traffic profile with a forward time window and refreshes the encoder periodically. It achieves a 1.19% gain at $C = 49$, though the optimal time-slicing configurations depend on the graph size. The graph-size-independent architecture introduces an adaptive pooling modification, which successfully mitigates the cross-size generalisation limitation with minimal quality degradation (at most 1.24%). Comparison against classical heuristics confirms that all best-performing neural configurations outperform Nearest Neighbour, Greedy Edge, NNR, NN+1, and 2-opt in solution quality at both graph sizes, while running approximately 97,000 times faster per instance than 2-opt at $C = 49$. The results demonstrate that lightweight architectural extensions can meaningfully improve neural routing performance on time-dependent problems.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Deep Reinforcement Learning for Combinatorial Optimization	2
1.3	Research Gaps and Contributions	3
	Gap 1: Lack of Awareness of the Tour Progress in the Decoder	3
	Gap 2: Absence of Classical Heuristic Knowledge in Neural Routing	4
	Gap 3: Static Encoder Consuming the Full Traffic History . .	5
1.4	Thesis Structure	5
2	Related Work	7
2.1	Classical TSP and Time-Dependent TSP	7
2.2	Classical Heuristics for the TDTSP	8
2.3	Meta-Heuristics for Dynamic and Time-Dependent TSP	9
2.4	Neural Combinatorial Optimization Foundations	9
2.5	Deep Reinforcement Learning for Dynamic and Time- Dependent Routing	11
2.6	Step-Aware and Phase-Dependent Decoding	12
2.7	Cost-Aware Mechanisms in Neural Routing	13
2.8	Encoder-Decoder Modifications and Re-Encoding Strategies .	13
2.9	REINFORCE with Baseline Strategies	14
2.10	Summary	15
3	Methodology	16
3.1	Overview of Approach	16
3.2	Problem Formulation and MDP Framework	18
	Time-Dependent Traveling Salesman Problem	18
	DRL Approach: MDP Formulation	20
	Policy Representation	21
	Learning Objective	22
3.3	Graph Attention Encoder	22

Encoder Inputs	22
Multi-Head Attention Process	23
Adaptation to the Deterministic TDTSP Setting	25
3.4 Decoder: Attention-Based Pointer	25
Stage 1: Dynamic Input Embedding	25
Stage 2: Context and Query Construction	27
Stage 3: Pointer Attention	27
3.5 Proposed Architectural Extensions	28
Step-Aware Decoder (Step-MLP)	28
Cost-Aware Gating and Heuristic Blending	31
Time-Sliced Traffic Encoding and Safe Refresh	32
3.6 Training Methodology	34
Data Generation and Preprocessing	34
Policy Gradient: REINFORCE Algorithm	36
Baseline Strategies	37
Training Process and Baseline Update	38
3.7 Graph-Size-Independent Architecture	40
Visited State Projection	40
Traffic Embedding Projection	42
3.8 Summary	44
4 Experiments	45
4.1 Experimental Setup	45
4.2 Baseline Model (M1) Performance	47
4.3 Step-MLP Variants	48
4.4 Batch Size and Training Stability	52
4.5 Hyperparameter Optimisation	52
4.6 Cost-Aware Gating	53
4.7 Time Slicing with Safe Refresh	55
4.8 Graph-Size-Independent Architecture	60
4.9 Beam Search	61
4.10 Comparison with Heuristic Baselines	61
4.11 Summary, Discussion, and Recommendation	63
Summary of Results	64
Overall Recommendation	65
5 Conclusion	66
5.1 Conclusion	66
5.2 Limitations	67
5.3 Future Work	68

List of Figures

1.1	Illustration of time-dependent travel times in TDTSP. The salesman at the depot (van) must visit customers A and B. Travel times (in minutes) vary based on departure time: 08:00 (rush hour), 09:00 (mid-morning), and 10:00 (off-peak). Red arrows indicate one direction of travel; black arrows indicate the reverse direction.	2
3.1	Full architecture of the proposed model. Components highlighted with red dashed boxes represent the proposed enhancements over the M1 baseline.	18
3.2	MDP framework for the TDTSP. The agent (M1 model) observes the current state, selects the next customer as the action, and receives the negative travel time as the reward.	21
3.3	Time slicing applied to the traffic profile. (a) Full-day representation with 12 bins of 2 hours each. (b) Sliced window of 6 bins (50% reduction) starting from the current time (8:00 AM).	33
3.4	Complete training workflow for the attention-based decoder with REINFORCE and baseline strategies. The dashed box indicates the baseline update logic applied at the end of each epoch.	39
4.1	Location map of the benchmark dataset (Beijing). The depot is located at the bottom-right of the map. Reproduced from (Zhang et al., 2023).	46
4.2	Mean validation cost $\pm$ standard deviation across three seeds (1234, 5678, 9012) over 300 epochs at $C = 49$ for Temp-MLP (solid blue) and Baseline (dashed orange). The shaded regions represent $\pm$ one standard deviation. The Temp-MLP mean remains consistently below the baseline throughout training, with a stable gap from approximately epoch 30 onwards.	50
4.3	Summary of decoder MLP variant performance at $C = 19$ and $C = 49$	51

4.4	Hyperparameter sweep results: Step-MLP hidden dimension (left), learning rate (centre), and learning rate decay (right). All experiments use $C = 20$, batch size 256, 50 epochs.	53
4.5	Fixed lambda sensitivity with NN heuristic ($C = 19$, seed 1234).	54
4.6	Mean test cost vs. window size W for $C = 19$, $C = 20$, and $C = 29$ (start bin = 3, one-time refresh, 50 epochs).	56

List of Tables

3.1	Step feature definitions for the Step-MLP.	29
3.2	Step-MLP feature presets.	30
3.3	Datasets used in training and evaluation.	36
3.4	Adaptive average pooling example for graph size = 19 ($\mathbb{R}^{400} \rightarrow \mathbb{R}^{128}$).	43
3.5	Adaptive average pooling example for graph size = 29 ($\mathbb{R}^{900} \rightarrow \mathbb{R}^{128}$).	43
4.1	Hyperparameters explored and selected values.	47
4.2	Baseline M1 performance across training epochs and graph sizes.	47
4.3	Feature preset comparison ($C = 19$, 50 epochs).	48
4.4	Leave-one-out ablation of v2_light ($C = 19$, 50 epochs).	49
4.5	Decoder MLP variant results ($C = 19$, 50 epochs).	49
4.6	Decoder MLP variant comparison ($C = 49$, 100 epochs, batch size = 256).	50
4.7	Batch size effect on test cost and training stability ($C = 49$, 100 epochs).	52
4.8	Selected hyperparameters ($C = 20$, Step-MLP v2_light, batch size 256).	53
4.9	Fixed lambda results by heuristic ($C = 19$, 100 epochs).	54
4.10	Cost-aware gating results ($C = 49$, 100 epochs, batch size 256, fixed $\lambda = 0.25$).	55
4.11	Best refresh strategy per graph size.	57
4.12	Time slicing validation ($C = 49$, $W = 3$, start bin = 3, 100 epochs).	57
4.13	Computational efficiency: time slicing vs. baseline.	58
4.14	Time slicing summary - best configuration per graph size.	58
4.15	Temp-MLP and Time Slicing combination results ($C = 49$, 100 epochs, batch size 256).	59
4.16	Within-size and cross-size performance (100 epochs, batch size 256).	60
4.17	Beam search results – solution quality ($B = 3$).	61

4.18	Solution quality and inference time comparison at $C = 19$ (mean $\pm$ SD across three seeds).	62
4.19	Solution quality and inference time comparison at $C = 49$ (mean $\pm$ SD across three seeds).	63

Chapter 1

Introduction

1.1 Background and Motivation

The classical Traveling Salesman Problem (TSP) is a well-known problem in many fields, including computer science, operations research, and applied mathematics. The main goal is to find the shortest path along which a salesman must visit each city once and return to the depot. TSP belongs to the class of NP-hard problems. In the real world, most domains that use TSP include logistics, supply chain management, manufacturing, telecommunications, and many more industries.

The classical TSP assumes that travel costs are static and symmetric. In reality, travel times between two locations can vary depending on the departure time due to factors such as traffic congestion, rush hours, weather conditions, maintenance, and others. This motivates the study of the Time-Dependent Traveling Salesman Problem (TDTSP), where the travel time for each edge depends on the departure time from its origin node.

Figure 1.1 illustrates this concept with a simple example. A salesman at the depot must visit two customers, A and B, and return to the depot. The three panels show how travel times between the same locations change throughout the morning. At 08:00 during rush hour, the travel time from the depot to customer A is 60 minutes (red arrow) while the route to B takes 65 minutes (black arrow). By 09:00, the travel time from A to B increases to 75 minutes due to shifting congestion patterns. At 10:00, as rush hour subsides, most travel times vary further. This temporal variation has important implications for route planning. A visiting sequence that is optimal when departing at 08:00 may be suboptimal when departing at 09:00 or 10:00. An effective routing strategy must therefore anticipate how traffic conditions will evolve during the tour, not merely react to current conditions.

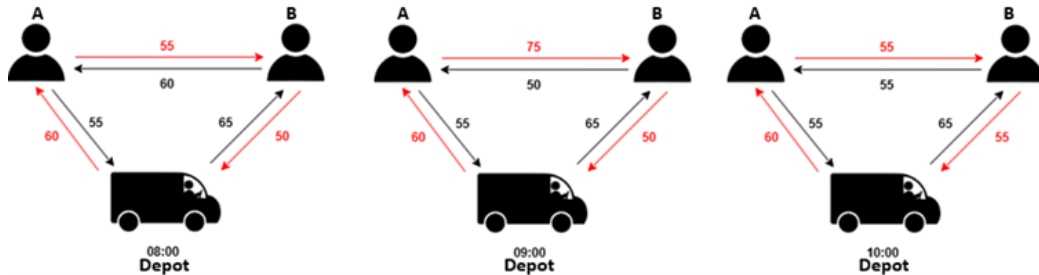


Figure 1.1: Illustration of time-dependent travel times in TDTSP. The salesman at the depot (van) must visit customers A and B. Travel times (in minutes) vary based on departure time: 08:00 (rush hour), 09:00 (mid-morning), and 10:00 (off-peak). Red arrows indicate one direction of travel; black arrows indicate the reverse direction.

One of the earliest studies on the TDTSP used integer programming to solve it (Picard and Queyranne, 1978). This foundational work inspired subsequent research in this area. Later, Malandraki and Daskin (1992) established a foundation for this problem class by proposing formulations for time-dependent vehicle routing problems (TDVRP) with step-function travel times. Most studies on the TDTSP assume the First-In-First-Out (FIFO) property. This property ensures that leaving later from a node cannot result in arriving earlier at the destination (no overtaking).

Heuristic methods such as time-expanded formulations, simulated annealing, ant colony optimization, and genetic algorithms can produce high-quality results, but solving the TDTSP is a computationally expensive task. This was demonstrated by Cordeau et al. (2014), who showed that even sophisticated branch-and-cut and branch-cut-and-price algorithms can only solve moderate-size instances to optimality.

1.2 Deep Reinforcement Learning for Combinatorial Optimization

In the past decade, deep reinforcement learning (DRL) and attention-based neural architectures have been used to solve routing problems. The Pointer Network is an encoder-decoder model that uses Sequence-to-Sequence (Seq2Seq) models to solve routing problems (Vinyals et al., 2015). Bello et al. (2017) replaced the Pointer Network’s supervised learning (which relies on labelled training data) with the REINFORCE algorithm using a simple exponential moving average baseline.

The introduction of the Transformer architecture by Vaswani et al. (2017) significantly revolutionized the field, as it replaced recurrent neural networks (RNNs) entirely with self-attention mechanisms. Kool et al. (2019) combined all of these innovations and produced the Attention Model (AM), which has become the standard baseline for solving routing problems. By following the footsteps of the Attention Model, Zhang et al. (2023) proposed a deep reinforcement learning (DRL) framework for variants of the time-dependent TSP. They proposed two model variants called M1 and M2 to handle the time dependent dynamic TSP. In a fixed customer base problem, all customers are known at the start of the tour and remain unchanged throughout the whole tour. In contrast, a dynamic customer base allows new customers to appear or existing customers to be removed during the tour. M1 is more suitable for a fixed customer base, while M2 is better suited for a dynamic customer pool. In this thesis, we use the M1 model as the baseline, as we solve the deterministic TDTSP with a fixed customer set (Zhang et al., 2023). Furthermore, the paper reports that M1 achieves more than 5% improvement over the M2 model for a fixed customer set. However, we identified several gaps in the M1 model that leave room for potential improvements.

1.3 Research Gaps and Contributions

This section identifies three significant gaps in the current state-of-the-art model M1, and for each gap, we describe the limitation, the importance of filling the gap, and our proposed solution.

Gap 1: Lack of Awareness of the Tour Progress in the Decoder

At each decoding step, the standard M1 decoder receives only the following information: (1) graph embedding, (2) depot embedding, (3) a binary visited mask indicating which nodes have been visited, (4) the embedding of the last-visited node, and (5) the current traffic conditions. Crucially, the decoder has no explicit signal indicating how far through the tour it has progressed. This lack of progress awareness may be problematic for the TDTSP. In TDTSP, decisions made at different stages of the route have fundamentally different characteristics and should therefore be treated differently. For example, a tour beginning in the morning must anticipate afternoon traffic patterns, while an evening tour may benefit from declining congestion as it progresses. Without an explicit step ratio or temporal progress signal, the decoder cannot differentiate early from late tour decisions.

Previous work by Wu et al. (2021) recognized this limitation and proposed augmenting the decoder with a Gated Recurrent Unit (GRU). At each decoding step t , the GRU takes the current context vector as input and combines it with its hidden state from the previous step (H_{t-1}), producing an enhanced context vector H_t that implicitly encodes the history of visited nodes and decisions made so far. While the GRU is effective, it can be computationally expensive, and it is therefore worthwhile to replace it with a lightweight MLP (Step-MLP) with tour-aware features (step ratio, last 3 visited nodes, visited node embeddings, etc.). We also conduct experiments by varying the position of the MLP layer:

- **Pre-attention MLP:** Nonlinear refinement of the query before attention computation.
- **Post-attention MLP:** Refinement of the glimpse vector after attention but before final logit computation.

Additionally, we improve accuracy by scaling the attention logits with a scaling factor (temperature scaling factor) learned by a lightweight MLP layer (temp-MLP).

Gap 2: Absence of Classical Heuristic Knowledge in Neural Routing

Classical routing heuristics (nearest neighbor, greedy cost minimization) often provide reasonable, interpretable, and computationally inexpensive solutions. Therefore, we incorporate this knowledge into the decoder and produce a hybrid approach.

Recently, researchers have begun exploring this direction. As a result, Ye et al. (2023) proposed DeepACO, where neural networks generate heuristic measures to guide Ant Colony Optimisation search. More recently, Huang et al. (2025) presented ReLD, which adds a logarithmic distance term to the attention logits:

$$p_i = \text{Softmax}\left(C \cdot \tanh\left(\frac{q_c^\top H_t}{\sqrt{d_h}} - \log(\text{dist}_i)\right)\right)_i \quad (1.1)$$

However, this approach has two main limitations for time-dependent routing problems. First, it uses static distance metrics (Euclidean distance) that are not suitable for TDTSP. Second, the heuristic integration is fixed (simple logarithmic transformation) without learnable parameters to adapt the heuristic influence based on problem state.

To address these limitations, this thesis proposes a Cost-Aware Gating mechanism that: (1) uses time-dependent travel costs rather than static distances as the heuristic signal, (2) employs a learnable gating parameter λ_{heur} that can adapt the heuristic influence, and (3) explores nonlinear transformations (piecewise-linear, exponential) of heuristic logits. We blend the heuristic logits as a bias into the attention logits via:

$$\ell_i^{\text{blended}} = \ell_i + \lambda_{\text{heur}} \cdot h_i(s_t) \quad (1.2)$$

where ℓ_i denotes the raw attention logit for candidate node i , $h_i(s_t)$ is the heuristic score, and λ_{heur} is a learnable parameter.

Gap 3: Static Encoder Consuming the Full Traffic History

The encoders of standard attention models, including our M1 baseline (Zhang et al., 2023), take the full-day traffic profile as input. However, in real-world dynamic routing problems, specific opening and closing times mean that feeding the full-day traffic profile wastes both computational resources and time. Therefore, we feed a time window, instead of the full history, into the encoder. We also refresh the encoder when the current decision time exceeds the encoded window boundary. We could not find any prior work in this area, and this thesis addresses this gap with the Time Slicing with Safe Refresh mechanism.

1.4 Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2: Related Work provides a comprehensive review of the literature on classical and time-dependent TSP, traditional heuristic methods, neural combinatorial optimization, and deep reinforcement learning for routing problems.

Chapter 3: Methodology discusses the technical details of our approach. First, we describe the baseline M1 model’s architecture, including the encoder-decoder attention model. Next, we discuss the implementation details of the three proposed enhancements (Step-MLP, the cost-aware gating mechanism, and time slicing with safe refresh). Finally, we describe the training procedure with the REINFORCE algorithm with rollout baseline.

Chapter 4: Experiments presents the end-to-end experimental setup and results. We describe the real-world Beijing dataset, baseline comparisons,

and evaluation metrics. We then present results for each proposed enhancement, including ablation studies and hyperparameter sensitivity analyses. We examine computational costs, training stability, and scaling behaviour from 19-node to 49-node instances.

Chapter 5: Conclusion summarises the contributions of this thesis and their implications for the broader field of neural combinatorial optimization.

Chapter 2

Related Work

This chapter reviews related work relevant to the deterministic Time-Dependent Traveling Salesman Problem (TDTSP) with a fixed customer set. We begin with classical TSP and TDTSP formulations, followed by classical heuristics and meta-heuristic methods. We then review foundational neural combinatorial optimization approaches, including Pointer Networks, the Transformer architecture, and the Attention Model. We proceed to discuss deep reinforcement learning methods designed for dynamic and time-dependent routing, with particular focus on the M1/M2 baseline models. Finally, we review step-aware decoding, cost-aware mechanisms, encoder re-encoding strategies, and REINFORCE training baselines that are directly relevant to the contributions of this thesis.

2.1 Classical TSP and Time-Dependent TSP

The classical TSP finds the shortest path through a set of cities, visiting each city exactly once before returning to the depot (starting point). TSP is an NP-hard combinatorial optimization problem (Papadimitriou, 1977). The standard formulation assumes travel costs are fixed and symmetric. In real life, problems in logistics, transportation systems, and delivery services regularly involve this type of TSP. However, travel times between locations depend on the departure time due to traffic congestion, maintenance, rush hours, or any other factors. This motivates the use of the TDTSP instead of the classical TSP.

One of the earliest studies on the TDTSP defined the cost as depending on the departure time and used integer programming to solve it (Picard and Queyranne, 1978). Later, the time-dependent vehicle routing problem (TDVRP) was proposed with a mixed integer linear programming formulation

and step-function travel times (Malandraki and Daskin, 1992).

A common assumption in the TDTSP is the First-In-First-Out (FIFO) property, which ensures that leaving later from a node cannot result in arriving earlier at the destination (no overtaking). A speed-based travel time model that satisfies the FIFO property was introduced (Ichoua et al., 2003).

Solving the TDTSP is computationally more complex than the classical TSP, as demonstrated by Cordeau et al. (2014), who showed that even sophisticated branch-and-cut and branch-cut-and-price algorithms can only solve moderate-size instances to optimality. The importance of time-dependent models in real-life transportation planning has been highlighted (Gendreau et al., 2015).

2.2 Classical Heuristics for the TDTSP

Before deep diving into advanced models, it is worth noting that classical local search heuristics can produce comparably good results on the TDTSP.

The Nearest Neighbor (NN) heuristic is one of the simplest solution heuristics for solving the TSP. Starting from the depot node, it selects the closest unvisited node until it reaches the last node and then returns to the depot. For time-dependent problems, NN naturally adapts (Malandraki and Daskin, 1992) and computational complexity remains $O(n^2)$. However, NN only considers the immediate next step without accounting for future consequences. **The NN+1 lookahead heuristic** mitigates this limitation by also considering how good the next move is and selecting the best node accordingly, though this increases computational complexity to $O(n^3)$ (Rosenkrantz et al., 1977). **The Nearest Neighbor Random (NNR)** heuristic, instead of always picking the minimum travel time node, assigns probability-based scores to the top- k candidate nodes (Malandraki and Dial, 1996).

The Greedy Edge heuristic is a classical construction heuristic that builds a tour by repeatedly selecting the shortest available edge in the graph, subject to two constraints: (1) no node may have degree greater than two, and (2) no subtour may be formed until all nodes have been included. Unlike the Nearest Neighbour heuristic, which builds a tour node by node from a starting point, Greedy Edge operates globally by sorting all edges by cost and greedily adding the cheapest feasible edge. For the TDTSP, edge costs are evaluated using the travel time at the current departure time via cubic spline interpolation. The computational complexity is $O(n^2 \log n)$ due to the initial edge sorting step.

The 2-opt local search heuristic (Croes, 1958) is one of the most effective methods for solving the static TSP by swapping two edges in a tour

to reduce the total length. While evaluating a swap in the static TSP requires only constant time, the TDTSP requires recomputing travel times for all subsequent nodes due to shifted departure times, increasing the computational complexity from $O(n^2)$ to $O(n^3)$ per full sweep (Cordeau et al., 2014; Ehmke et al., 2018; Vander Wiel and Sahinidis, 1995). In this thesis, the 2-opt heuristic is initialised with a Nearest Neighbour tour and iteratively improves it by swapping pairs of edges until no further improvement is found.

2.3 Meta-Heuristics for Dynamic and Time-Dependent TSP

Meta-heuristic techniques such as Simulated Annealing (Schneider, 2003), Ant Colony Optimization (Mavrovouniotis and Yang, 2013), and Genetic Algorithms (Haghani and Jung, 2005) are well-known for solving time-dependent and dynamic routing problems. These techniques are capable of producing high-quality solutions. However, in real-time applications, they face difficulties due to their iterative nature, which leads to significant computation time per instance, lack of generalisation, and sensitive parameter tuning. These limitations motivate the investigation of learning-based mechanisms that reduce computational overhead through offline training.

2.4 Neural Combinatorial Optimization Foundations

In recent years, applying deep learning methods to combinatorial problems has seen rapid progress. This section reviews key developments from sequence-to-sequence models to Transformer-based attention models, graph neural networks, and symmetry-aware MDP formulations.

Pointer Networks and Sequence-to-Sequence models. An encoder-decoder model for machine translation was built using Sequence-to-Sequence (Seq2Seq) models (Sutskever et al., 2014). The main limitation of this model is that the decoder outputs from a fixed vocabulary, making it unsuitable for the TSP as the problem does not have a fixed dictionary. To address this limitation, the Pointer Network was introduced to solve combinatorial optimization problems using an attention mechanism (Vinyals et al., 2015). The encoder recurrent neural network (RNN) embeds the sequence of customer coordinates, and a decoder RNN produces the tour by pointing to input positions at each step. The Pointer Network’s supervised learning was replaced

with the REINFORCE algorithm using a simple exponential moving average baseline, showing that reinforcement learning can outperform supervised learning on the TSP instances (Bello et al., 2017).

The Transformer architecture. Since Pointer Networks use RNNs (LSTM/GRU) in both the encoder and decoder, they suffer from limited parallelization and vanishing gradient issues. As a solution, the Transformer architecture was introduced, which completely replaced RNNs with multi-head self-attention mechanisms (Vaswani et al., 2017). The key components are scaled dot-product attention, multi-head attention, positional encodings, and feed-forward sublayers with residual connections and layer normalisation. The Transformer enabled parallel processing and is well suited for capturing long-term dependencies.

The Attention Model. The ideas discussed above were developed separately and had not been combined for routing problems. The insights from Pointer Networks, REINFORCE training, and the Transformer architecture were combined into the Attention Model (AM), which has become the standard baseline for solving routing problems (Kool et al., 2019). The proposed model consists of a one-time encoder and an autoregressive decoder. The encoder takes the city coordinates as inputs and its internal multi-head self-attention mechanism produces node embeddings and a graph embedding as output. The decoder creates a context vector using the encoder output, the first visited node embedding, and the last visited node embedding. Using this context vector and the node embeddings, the decoder applies a self-attention mechanism and generates the sequential output. The decoder is trained using the policy gradient method (REINFORCE) with a rollout baseline on large batches of randomly generated instances.

Graph Neural Networks for routing. Graph Neural Networks (GNNs) offer an alternative encoder paradigm that directly exploits graph structure. Structure2Vec-DQN was proposed, combining Structure2Vec embeddings with deep Q-learning for greedy node-by-node construction on the TSP and other graph problems (Dai et al., 2017). Graph Convolutional Networks were used to predict edge probability heatmaps for the TSP, followed by beam search (Joshi et al., 2019). GNNs can also adapt to the TDTSP, but in this thesis we mainly focus on the Transformer-based Attention Model paradigm (Kool et al., 2019; Zhang et al., 2023).

BQ-Transformer and symmetry-aware MDP formulations. All of the models discussed (Pointer Networks, the Attention Model, and GNN-based approaches) so far share a key limitation: the MDP state grows at each step. The model must carry the full history of visited nodes, even though this history is no longer needed to decide the next move. To address this, the BQ-Transformer was introduced at NeurIPS 2023 (Drakulic et al., 2023).

Its main innovation is that at each step, only the remaining unvisited nodes matter, not the full history. Rather than encoding the full instance once, the same Transformer model is re-applied to the remaining subproblem at every construction step. The policy is trained using imitation learning on near-optimal solutions of small instances rather than reinforcement learning, yet achieves state-of-the-art generalisation to much larger instances. This idea of re-encoding the remaining subproblem at each step directly motivates the re-encoding strategies proposed in this thesis (Section 3.5).

Decoding strategies in neural routing. During both training and evaluation of neural routing models, the policy network must decode a complete tour from the learned probability distributions. The choice of decoding strategy significantly impacts both solution quality and computational cost.

Sampling: During training with the REINFORCE algorithm, tours are typically generated by sampling from the policy distribution π_θ at each decoding step.

Greedy Decoding: At evaluation time, greedy decoding selects the highest-probability action at each step (argmax) (Kool et al., 2019).

Beam Search: Beam search maintains k candidate partial solutions at each step, expanding each by the top- k actions and keeping the k best complete solutions. While computationally more expensive, it can find higher-quality solutions by exploring multiple promising paths simultaneously (Choo et al., 2022).

2.5 Deep Reinforcement Learning for Dynamic and Time-Dependent Routing

The sections above discussed static routing problems such as the TSP and the VRP without much attention to time dependence. This section directly discusses dynamic and time-dependent routing problems.

The M1/M2 baseline models. A deep reinforcement learning (DRL) framework (Zhang et al., 2023) was proposed by extending the Attention Model of Kool et al. (2019). The proposed model is suited for the Dynamic Traveling Salesman Problem (DTSP) and a dynamic pickup-and-delivery problem (DPDP). In these problems, travel times depend on the departure time and new customers can appear or disappear during the tour. Their DRL framework treats the routing problem as a Markov decision process and trains an attention-based encoder-decoder policy using the REINFORCE algorithm.

The baseline paper (Zhang et al., 2023) introduced two attention-based encoder-decoder models called M1 and M2. The M1 model follows an encoder-

once and decoder-online design. The encoder processes node identities, traffic patterns, and demand (only for DPDP) and produces node embeddings and a graph embedding. The decoder takes five inputs: (1) $w(t)$, the current traffic embedding; (2) $v(t)$, the visited status embedding; (3) $\bar{h}$, the graph embedding; (4) h_0 , the depot embedding; and (5) h_l , the last visited node embedding. By concatenating these five embeddings, the decoder generates the context vector and the attention mechanism selects the next node to visit. M1 is computationally efficient and performs well on a fixed customer base.

The M2 model is designed to handle dynamic customer pools and re-encodes the state after each visit. The key difference from M1 is that the M2 training process is divided into two stages: a pretraining stage that trains two separate sub-models called Node2Vec (for node positions) and Traffic2Vec (for traffic patterns), and a fine-tuning stage that combines them. This strategy is adopted because running M2 end-to-end is computationally expensive.

Both models use DRL for training and solve the time-dependent TSP for both deterministic and stochastic travel times. Experimental results show that M1 achieves more than 5% improvement for a fixed customer set compared to M2. Therefore, we plan to use the M1 model as the baseline for this thesis.

Recent DRL approaches for time-dependent routing. Dynamic Graph Temporal Attention (DGTA) was introduced for the dynamic TSP with both time-dependent and stochastic travel times (Chen et al., 2025). They use a dynamic encoder with a multi-head attention mechanism to address the dynamic customer pool. This means their work does not focus on a fixed customer set and addresses stochastic rather than deterministic travel times.

The RNN²-AM (RNN-squared Attention Model) was proposed for the Time-Dependent TSP with Time Windows (TDTSP_{TW}) (Wu et al., 2021). They use an LSTM for traffic pattern encoding at the top of the encoder and a GRU to enhance the context vector. This paper is closely related to our baseline M1 approach, but it uses windowed timings in its problem formulation.

2.6 Step-Aware and Phase-Dependent Decoding

One of the main enhancements in this thesis is the step-aware decoder. The decoder context vector in (Wu et al., 2021) uses the same five components as in our M1 baseline model, and once the context vector is created, it is passed through a GRU layer. This generates an enhanced context vector:

$$H_t = \text{GRU}(C_t, H_{t-1}) \quad (2.1)$$

where the GRU hidden state H_{t-1} captures sequential tour history, ready for the attention computation. In their paper, they experimented with replacing the GRU with an MLP layer, which reduced performance. The results show that while the GRU performs well, it incurs significant computational cost, making it worthwhile to explore a lighter alternative.

2.7 Cost-Aware Mechanisms in Neural Routing

While classical heuristic models (nearest neighbor, greedy cost minimization) can generate reasonably good results for routing problems, they do not leverage learned representations. Therefore, we integrate this classical knowledge into the M1 baseline model.

DeepACO was proposed, where neural networks generate heuristic measures (pheromone and heuristic information) that guide the Ant Colony Optimisation search (Ye et al., 2023). In DeepACO, the probability of selecting the next node is computed as the product of a learned ACO update and a learned heuristic bias. More recently, Huang et al. (2025) presented ReLD, which adds a logarithmic distance term to the attention logits before softmax (see Equation 1.1). However, this approach uses static distance metrics and fixed heuristic transformations, limiting their applicability to TDTSP. This thesis proposes a Cost-Aware Gating mechanism that addresses these limitations using time-dependent travel costs and a learnable gating parameter. The mechanism is detailed in Section 3.5.

2.8 Encoder-Decoder Modifications and Re-Encoding Strategies

We propose a refreshing mechanism in this thesis and therefore it is essential to review previous re-encoding approaches.

A deep reinforcement learning algorithm using a Dynamic Attention Model for vehicle routing problems was proposed (Peng et al., 2019). Since the vehicle has limited capacity, it must return to the depot and reload before the next route. Due to this return-to-depot behaviour, the encoder needs to run again with a different set of unvisited nodes (sub-problem). They found that dynamic encoding increased the computational cost significantly.

Step-wise deep learning models for solving routing problems were proposed (Xin et al., 2021b). They removed the just-visited node from the graph and re-encoded at every step. This approach is justified by Bellman’s Principle of Optimality, which states that at each step, the remaining problem is itself an optimal sub-problem. This approach is also computationally expensive.

The Multi-Decoder Attention Model was introduced for problems such as the VRP (Xin et al., 2021a). This mechanism periodically re-embeds node features during decoding (every p steps) to remove information about already-visited nodes.

The M1 model was trained and tested on fixed customer sets of size $C = 4, 9$, and 19 , sampled from a pool of 100 customers (Zhang et al., 2023). As a result, the encoder–decoder model is dependent on the graph size, which means the model needs to be retrained every time the customer base changes. This dependency is challenging in real-world applications, as the customer base varies from time to time and therefore a more generalised model is needed. Adaptive average pooling (AdaptiveAvgPool) addresses this by mapping any variable-length input to a fixed-size output before the linear projection, without discarding information through padding. This technique is well established in computer vision for handling variable input sizes, originally introduced by He et al. (2015) through Spatial Pyramid Pooling (SPP-net), which demonstrated that fixed-length representations can be generated regardless of input size or scale.

2.9 REINFORCE with Baseline Strategies

Since routing problems do not have labelled data, researchers use reinforcement learning in their training process. The policy gradient estimates are computed using sampled tours:

$$\mathcal{L}_{\text{REINFORCE}} = \frac{1}{B} \sum_{i=1}^B (C_i - b_i) \log p_{\theta}(\pi_i \mid x_i) \quad (2.2)$$

where C_i is the tour cost, b_i is a baseline, and B is the batch size.

An exponential moving average of batch costs was used as the baseline (Bello et al., 2017). Due to its slow convergence, Kool et al. (2019) introduced the greedy rollout baseline. This method saves a frozen copy of the best-performing model at each epoch and uses it to produce solutions via greedy decoding (more details in Chapter 3). The baseline is updated only when the current model outperforms it according to a paired t -test. This approach yields faster convergence and better final solutions than other methods.

2.10 Summary

Throughout this chapter, we have discussed previous approaches for the classical TSP and the time-dependent Traveling Salesman Problem, ranging from classical heuristics to modern attention-based deep reinforcement learning methods. The M1 model of Zhang et al. (2023) provides a strong foundation for the deterministic TDTSP with a fixed customer base. Building upon this foundation, the following chapter presents our methodology for addressing the limitations identified in Section 1.3.

Chapter 3

Methodology

This chapter presents the methodology for improving the deterministic Time-Dependent Traveling Salesman Problem (TDTSP) with a fixed customer set. We begin with an overview of the approach (Section 3.1), followed by a formal problem formulation and Markov Decision Process (MDP) framework (Section 3.2). We then describe the baseline encoder-decoder architecture (Sections 3.3–3.4) and the three proposed architectural extensions (Section 3.5). Finally, we detail the training methodology (Section 3.6).

3.1 Overview of Approach

This research focuses on the deterministic TDTSP with a static customer base. Travel time depends on the departure time and there is no stochasticity in the travel times (only deterministic behaviour). A Markov Decision Process (MDP) is used to describe the problem, where the model chooses the next customer (node) sequentially, one at a time. The goal is to minimise the total tour time.

We adopt the M1 model of Zhang et al. (2023) as the baseline, since it performs well on a static customer base. The M1 model is built on an encoder-decoder architecture. The encoder takes node identities and traffic time profiles, passes them through a linear projection layer and several Multi-Head Attention (MHA) layers, and generates a set of node embeddings and a global graph embedding. The decoder then generates the tour sequentially. To select the next node, the decoder uses the node embeddings, the graph embedding, the visited state, and the current traffic conditions.

Three architectural improvements are proposed to address the limitations identified in Section 1.3:

1. **Step-Aware Decoder via Step-MLP:** The standard decoder treats

all decoding steps identically using only visited node information and current traffic conditions, without any awareness of the tour progress. To address this limitation, we add a lightweight multi-layer perceptron (Step-MLP) that takes simple step-based features (step ratio, last-visited nodes, visited node embeddings, etc.) and produces a context nudge added to the decoder query, as well as an optional learned temperature that scales the attention logits.

2. **Cost-Aware Gating:** We inject classical routing heuristic knowledge into the decoder by combining heuristic logits with the raw attention logits. The classical heuristics considered are: linear travel time (Nearest Neighbour), the NN+1 two-step lookahead method, and the Nearest Neighbour Random (NNR) heuristic. A gating mechanism linearly blends the learned attention logits with the heuristic logits.
3. **Time Slicing with Safe Refresh:** Instead of feeding the full-day traffic profile into the encoder, we feed only a forward time window from the current decision time. A safe refresh mechanism re-encodes the traffic profile whenever the current time approaches or exceeds the window boundary.

The policy is trained using the REINFORCE algorithm with rollout baseline, where the baseline model is updated by comparing with the current model via a paired t -test. During training, cubic spline coefficients fitted to real Beijing traffic data are used as inputs rather than raw travel times. Nodes are selected randomly per instance to form sub-problems of variable graph size.

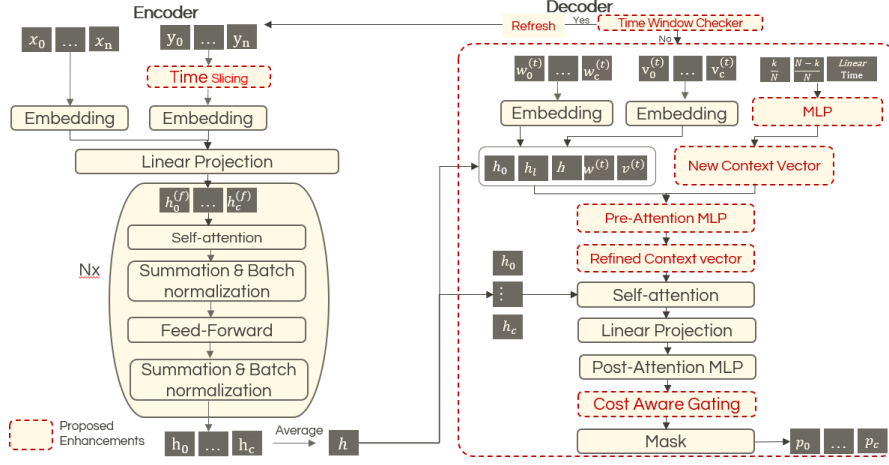


Figure 3.1: Full architecture of the proposed model. Components highlighted with red dashed boxes represent the proposed enhancements over the M1 baseline.

3.2 Problem Formulation and MDP Framework

This section presents the formal definition of the deterministic TDTSP and its formulation as a Markov Decision Process (MDP). The MDP framework enables the application of deep reinforcement learning methods, where a neural network policy learns to make sequential routing decisions.

Time-Dependent Traveling Salesman Problem

The TDTSP is defined on a complete directed graph $G = (V, E)$, where $V = \{0, 1, 2, \dots, n\}$ is the set of nodes (customers) and $E = V \times V$ is the set of directed edges representing travel times. In the classical TSP, edge costs are constant and mostly symmetric. In the TDTSP, however, the travel time from node i to node j depends on both the edge and the departure time $d_{ij}(t)$, where t denotes the departure time from node i .

The salesman starts at the depot (node 0) at time $t = 0$, must visit each customer exactly once, and return to the depot. The main objective is to find a tour $\pi = (\pi_0, \pi_1, \dots, \pi_n, \pi_{n+1})$ that minimises the total travel time, where $\pi_0 = \pi_{n+1} = 0$ (depot) and $(\pi_1, \dots, \pi_n)$ are the selected customer nodes. The objective function is:

$$\min_{\pi} \sum_{k=0}^n d_{\pi_k, \pi_{k+1}}(t_{\pi_k}) \quad (3.1)$$

where t_{π_k} is the arrival time at node π_k , computed recursively as:

$$t_{\pi_0} = 0, \quad t_{\pi_{k+1}} = t_{\pi_k} + d_{\pi_k, \pi_{k+1}}(t_{\pi_k}) \quad \text{for } k = 0, 1, \dots, n \quad (3.2)$$

Travel times are represented using cubic spline interpolation, which provides smooth, continuous estimates between discrete time bins. For edge (i, j) at departure time t , the travel time $d_{ij}(t)$ is computed using four cubic spline coefficients fitted to the raw data. The 24-hour day is divided into $T = 12$ bins of 2 hours each, and the fractional position $z \in [0, 1/T)$ within the current bin determines the interpolated value. Full details of the travel time computation and data preprocessing are provided in Section 3.6.

FIFO Property. The First-In-First-Out (FIFO) property is a standard assumption in TDTSP, meaning that departing later cannot result in arriving earlier. Formally, for any edge (i, j) and departure times $t_1 < t_2$, the FIFO property requires:

$$t_1 + d_{ij}(t_1) \leq t_2 + d_{ij}(t_2) \quad (3.3)$$

In other words, if Person A departs at time t_1 and Person B departs later at time t_2 , then Person A must arrive no later than Person B.

Why FIFO Matters. The FIFO property is essential for meaningful route optimisation. Without FIFO, a model might suggest “wait here and depart later to arrive earlier”, which is a physically impossible strategy in real traffic. FIFO ensures that routing decisions are realistic and that the optimal solution represents a genuinely efficient route rather than an artifact of inconsistent travel times.

Guaranteeing FIFO in This Work. The Beijing dataset used in this thesis is derived from real-world traffic measurements collected by Jingdong Logistics. Real-world traffic patterns inherently satisfy FIFO because travel time variations reflect gradual congestion changes rather than sudden discontinuities. Vehicles travelling the same route cannot overtake simply by departing later. Additionally, the cubic spline interpolation used to smooth travel times between discrete bins (see Section 3.6) preserves continuity and avoids sudden drops in travel time that could violate FIFO. While we do not explicitly verify Equation 3.3 for every edge and time point, prior work using this dataset has adopted the same assumption (Zhang et al., 2023; Ichoua et al., 2003).

DRL Approach: MDP Formulation

We formulate the TDTSP as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ) and solve it using a deep reinforcement learning agent. The framework has four main components:

State Space (S). A state s_k at decision step k comprises all information needed to make the next routing decision:

- *Static attributes:* node identities and traffic patterns (cubic spline coefficients for all edges and time bins).
- *Dynamic attributes:* current node position π_k , current time t , current traffic condition $w(t)$, and visited status $v^{(k)}$.

Action Space (A). At each step k , the action a_k is the index of the next node to visit. The action space is state-dependent, containing only unvisited nodes:

$$\mathcal{A}(s_k) = \{j \in V : v_j^{(k)} = 0\} \quad (3.4)$$

Agent. The policy $\pi_\theta(a_k \mid s_k)$ is parameterised by the M1 encoder-decoder architecture and outputs a probability distribution over the action space $\mathcal{A}(s_k)$.

Reward (R). The reward is the negative travel time incurred by the transition, converting the cost minimisation objective into reward maximisation:

$$R(s_k, a_k) = -d_{\pi_k, a_k}(t) \quad (3.5)$$

where $d_{\pi_k, a_k}(t)$ is the travel time from the current node π_k to the selected node a_k at departure time t .

The interaction between state, agent, action, and reward is illustrated in Figure 3.2.

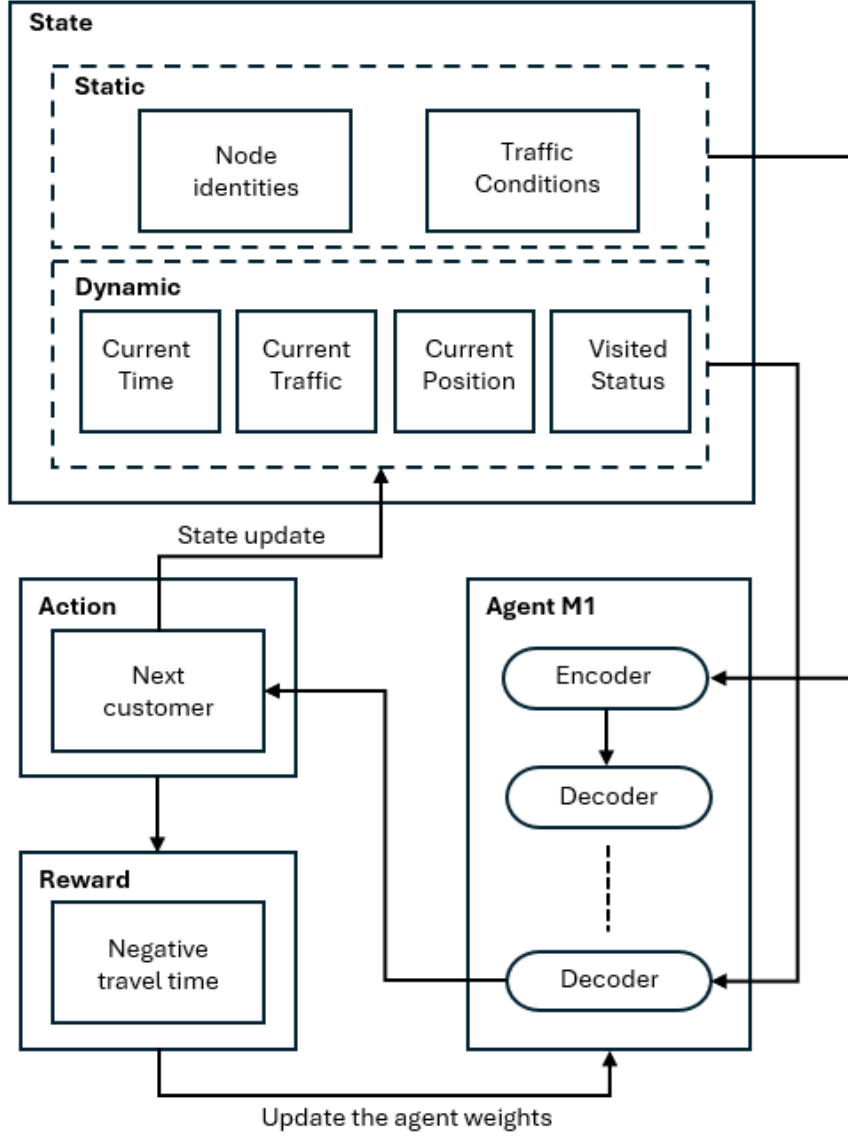


Figure 3.2: MDP framework for the TDTSP. The agent (M1 model) observes the current state, selects the next customer as the action, and receives the negative travel time as the reward.

Policy Representation

The policy $\pi_{\theta}(a_t | s_t)$ defines a probability distribution over actions given the current state, parameterised by neural network weights θ . In this thesis, the policy is represented by an attention-based encoder-decoder architecture (M1

model), detailed in Sections 3.3 and 3.4.

The policy factorises autoregressively over the sequence of decisions:

$$p_{\theta}(\pi \mid s) = \prod_{k=1}^N p_{\theta}(\pi_k \mid s, \pi_1, \dots, \pi_{k-1}) \quad (3.6)$$

where $\pi = [\pi_1, \pi_2, \dots, \pi_N]$ is the complete tour sequence, N is the number of nodes (graph_size + 1, including depot), θ represents the learnable parameters, and each $p_{\theta}(\pi_k \mid \cdot)$ is the probability distribution over unvisited nodes at step k .

Action Selection. During training, actions are sampled from the policy distribution to enable exploration:

$$a_t \sim \pi_{\theta}(\cdot \mid s_t) \quad (3.7)$$

During evaluation, greedy decoding is used to obtain deterministic solutions:

$$a_t = \arg \max_{a \in \mathcal{A}(s_t)} \pi_{\theta}(a \mid s_t) \quad (3.8)$$

Learning Objective

The goal of training is to find policy parameters θ^* that maximise the expected cumulative return (i.e., minimise expected total travel time):

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{\pi \sim p_{\theta}(\cdot \mid s)} [G(\pi, s)] \quad (3.9)$$

where $\mathcal{D}$ is the distribution of problem instances and $G(\pi, s)$ is the return (negative total travel time) for tour π on instance s . This objective is optimised using the REINFORCE algorithm with a rollout baseline, described in detail in Section 3.6.

3.3 Graph Attention Encoder

The encoder converts a TDTSP instance into a set of learned node embeddings and a global graph embedding. The encoder runs once per problem instance and produces representations used by the decoder at every decoding step.

Encoder Inputs

For each customer i in the problem instance, the encoder requires two types of input:

- **Node identities:** Each node is represented by a one-hot encoded positional vector $\mathbf{e}_i \in \mathbb{R}^{100}$, with a single 1 at the city index. It is then embedded into a 128-dimensional node-identity embedding:

$$x_i = W_{\text{id}} \mathbf{e}_i + b_{\text{id}}, \quad \text{where } W_{\text{id}} \in \mathbb{R}^{128 \times 100}, b_{\text{id}} \in \mathbb{R}^{128} \quad (3.10)$$

- **Traffic data:** For each node i in the problem instance, the encoder requires travel time information to all other nodes across all time bins. The raw traffic data is stored as cubic spline coefficients (see Section 3.6 for preprocessing details). For the encoder input, we extract the constant coefficient ($c_{i,j,b}^0$) for all outgoing edges from node i across all $T = 12$ time bins. Since there are $N_{\text{total}} = 100$ potential destination nodes and 12 time bins, this yields a flat vector of $100 \times 12 = 1,200$ values per source node:

$$\text{vec}(D_i) = [c_{i,0,0}^0, c_{i,0,1}^0, \dots, c_{i,0,11}^0, c_{i,1,0}^0, \dots, c_{i,99,11}^0] \in \mathbb{R}^{1200} \quad (3.11)$$

where $c_{i,j,b}^0$ is the constant spline coefficient for edge (i, j) at time bin b . This vector captures the base travel time pattern from node i to all destinations across the day. The vector is then projected to the embedding dimension:

$$y_i = W_{\text{traffic}} \text{vec}(D_i) + b_{\text{traffic}} \quad (3.12)$$

where $W_{\text{traffic}} \in \mathbb{R}^{128 \times 1200}$ and $b_{\text{traffic}} \in \mathbb{R}^{128}$.

The two components are concatenated and passed through a linear projection to produce the initial encoder embedding:

$$h_i^{(0)} = W_{\text{init}}[x_i; y_i] + b_{\text{init}}, \quad \text{where } W_{\text{init}} \in \mathbb{R}^{128 \times 256}, b_{\text{init}} \in \mathbb{R}^{128} \quad (3.13)$$

where $W_{\text{init}} \in \mathbb{R}^{d_{\text{embed}} \times d_{\text{node}}}$, with $d_{\text{embed}} = 128$ (embedding dimension) and $d_{\text{node}} = 256$ (concatenated node and traffic embedding dimension). This yields the input tensor $H^{(0)} \in \mathbb{R}^{(n+1) \times d_{\text{embed}}}$.

Multi-Head Attention Process

The initial embeddings $H^{(0)}$ are processed through $N = 2$ encoder layers, each with Multi-Head Attention (MHA) using $M = 8$ heads.

Each encoder layer takes an input embedding matrix $H \in \mathbb{R}^{(n+1) \times d_{\text{embed}}}$ and applies the following transformations. First, multi-head self-attention is applied with a residual connection:

$$H' = H + \text{MHA}(H) \quad (3.14)$$

The result is normalised using batch normalisation:

$$H'' = \text{BatchNorm}(H') \quad (3.15)$$

Next, a position-wise feed-forward network is applied with another residual connection:

$$H''' = H'' + \text{FFN}(H'') \quad (3.16)$$

Finally, batch normalisation is applied again:

$$H^{\text{out}} = \text{BatchNorm}(H''') \quad (3.17)$$

This sequence is repeated for each of the $N = 2$ encoder layers, with the output H^{out} of one layer serving as the input H to the next.

Self-Attention. The attention mechanism computes:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V \quad (3.18)$$

where Q, K, V are linear projections of the input, $d_k = d_{\text{embed}}/h = 16$ is the scaling factor (dimension of keys per head), and the softmax converts dot-product scores to attention weights. All weights are initialised from a uniform distribution over $[-\text{stdv}, \text{stdv}]$ where $\text{stdv} = 1/\sqrt{d_{\text{last}}}$.

Multi-Head Attention. The outputs of all heads are concatenated and linearly combined:

$$\text{MHA}(H) = [\text{head}_0 \parallel \text{head}_1 \parallel \dots \parallel \text{head}_7] W_O \quad (3.19)$$

Residual Connection. The original input is added back to each sublayer output, facilitating gradient flow and preserving the original data.

Batch Normalisation. After each sublayer:

$$\text{BatchNorm}(x) = \gamma \cdot \frac{x - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}} + \beta \quad (3.20)$$

Position-wise Feed-Forward Network. Each node embedding is independently transformed to introduce non-linearity and additional modelling capacity:

$$\text{FFN}(x) = \max(0, xW_1 + b_1) W_2 + b_2 \quad (3.21)$$

Graph Embedding (Mean Pooling). After all encoder layers, the global graph embedding is computed as the mean of all node embeddings:

$$g = \frac{1}{n} \sum_{i=0}^{n-1} h_i \quad (3.22)$$

Adaptation to the Deterministic TDTSP Setting

The M1 model was originally designed for the TDTSP with both deterministic and stochastic travel times. In this thesis, we focus exclusively on the deterministic case. To obtain deterministic travel times, noise is omitted from the travel time model.

The Beijing dataset provides discrete travel time measurements at 2-hour intervals throughout the day (12 time bins). To enable travel time computation at any departure time, cubic splines are fitted to these measurements, producing four coefficients per edge per bin. The encoder uses the base coefficient (c_0) as input features, while the decoder computes precise interpolated travel times using all four coefficients. Full details of the travel time representation and preprocessing are provided in Section 3.6.

In the original M1 architecture, the encoder runs once per problem instance and remains fixed during decoding, while the decoder runs at every node selection step. With the time-slicing extension proposed in Section 3.5, the encoder may be refreshed multiple times within a single tour when the current decision time exceeds the encoded time window.

3.4 Decoder: Attention-Based Pointer

The decoder solves the TDTSP by sequentially selecting the next node to visit. Each decision is autoregressive, depending on all previous decisions. The probability of choosing the next node is conditioned on the current state s (traffic conditions, remaining customers, visited mask, current time) and all previously selected nodes $\pi_1, \dots, \pi_{k-1}$.

At each step t , the decoder receives node embeddings $\{h_i\}$, the graph embedding h_{graph} , current travel time w^t , and visited state v^t , and outputs a probability distribution $\pi_\theta(\cdot | s_t)$. The decoder operates in three stages.

Stage 1: Dynamic Input Embedding

Current traffic condition ($w(t)$). At each decoding step k , the decoder computes the travel time for all ordered pairs of nodes at the current normalised time $t \in [0, 1)$. For each pair (i, j) , the travel time is computed using cubic spline interpolation:

$$d_{ij}(t) = c_{ij,b}^0 + c_{ij,b}^1 z + c_{ij,b}^2 z^2 + c_{ij,b}^3 z^3 \quad (3.23)$$

where $b = \lfloor t \cdot T \rfloor$ is the current time bin index, $(c_{ij,b}^0, c_{ij,b}^1, c_{ij,b}^2, c_{ij,b}^3)$ are the precomputed spline coefficients specific to edge (i, j) at bin b , and $z =$

$(t \cdot T - \lfloor t \cdot T \rfloor) / T \in [0, 1/T)$ is the fractional position within the bin, normalised by T (see Section 3.6 for details).

Stability clamp. Cubic interpolation can produce extreme values near bin boundaries. To prevent numerical instability, the interpolated travel time is clamped to a reasonable range. Let $\tau = \lfloor t \cdot T \rfloor$ be the current time bin index and $\tau^+ = (\tau + 1) \bmod T$ be the next bin index. For each edge (i, j) , define c_0 as the constant spline coefficient at bin τ and c_0^+ as the constant spline coefficient at bin τ^+ . The clamping bounds are:

$$d_{\min} = 0.05 \cdot (c_0 + c_0^+) \quad (3.24)$$

$$d_{\max} = 5 \cdot (c_0 + c_0^+) \quad (3.25)$$

This constrains the travel time to lie within $[5\%, 500\%]$ of the combined reference from adjacent bins. The clamped travel time is:

$$d_{ij}(t) \leftarrow \text{clip}(d_{ij}(t), d_{\min}, d_{\max}) \quad (3.26)$$

Traffic matrix assembly. The clamped travel times for all ordered pairs are assembled into a full traffic matrix:

$$W(t) \in \mathbb{R}^{n \times n} \quad (3.27)$$

where $W(t)_{ij} = d_{ij}(t)$ is the clamped travel time from node i to node j at time t . Importantly, this is the complete pairwise traffic matrix, not only the travel times from the current node π_k to each candidate (which would be a single row of $W(t)$). By using the full matrix, the model has access to the global traffic state at time t . This matrix is flattened row-by-row into a vector:

$$w(t) = \text{vec}(W(t)) \in \mathbb{R}^{n^2} \quad (3.28)$$

Traffic embedding. The traffic vector is projected into the model’s latent space:

$$\tilde{w}(t) = W_w \cdot w(t) + b_w \quad (3.29)$$

where $W_w \in \mathbb{R}^{d_h \times n^2}$ and $b_w \in \mathbb{R}^{d_h}$. The resulting embedding $\tilde{w}(t) \in \mathbb{R}^{d_h}$ captures the current traffic conditions across the entire graph.

Visited state / feasibility mask ($v^t \in \{0, 1\}^{n+1}$). The visited state is a Boolean vector of the graph size, converted to floating-point values and projected using a size-aware linear layer (no padding or pooling):

$$\bar{v}(t) = W_v v(t) + b_v \in \mathbb{R}^{128}, \quad W_v \in \mathbb{R}^{128 \times (n+1)} \quad (3.30)$$

The visited mask is updated after each node selection.

Stage 2: Context and Query Construction

A context vector c_t is constructed by concatenating five sources of information: (1) the embedded travel-time vector $\tilde{w}(t)$, (2) the embedded visited mask $\bar{v}(t)$, (3) the global graph embedding h_{graph} , (4) the depot embedding h_0 , and (5) the last-visited node embedding h_{i_t} :

$$c_t = [\tilde{w}(t) \parallel \bar{v}(t) \parallel h_{\text{graph}} \parallel h_0 \parallel h_{i_t}] \quad (3.31)$$

The attention query is derived from the context vector:

$$q_t = W_q c_t \in \mathbb{R}^{d_{\text{att}}}, \quad W_q \in \mathbb{R}^{d_{\text{att}} \times d_{\text{embed}}}, \quad d_{\text{att}} = d_{\text{embed}} = 128 \quad (3.32)$$

Stage 3: Pointer Attention

In Stage 3, the attention query and encoder node embeddings are combined to compute a probability distribution over the unvisited nodes.

1. Node Embedding Projections. Each encoder embedding $h_i \in \mathbb{R}^{128}$ is projected into three components:

$$\begin{bmatrix} k_i^{\text{glimpse}}, v_i^{\text{glimpse}}, k_i^{\text{logit}} \end{bmatrix} = W_{\text{node}} h_i + b_{\text{node}}, \quad W_{\text{node}} \in \mathbb{R}^{384 \times 128} \quad (3.33)$$

giving glimpse keys $k_i^{\text{glimpse}} \in \mathbb{R}^{128}$, glimpse values $v_i^{\text{glimpse}} \in \mathbb{R}^{128}$, and logit keys $k_i^{\text{logit}} \in \mathbb{R}^{128}$.

2. Multi-Head Attention for Glimpse Computation. The query, glimpse keys, and glimpse values are split into $h = 8$ heads with $d_k = 128/8 = 16$ per head. For each head $m \in \{1, \dots, 8\}$, compatibility scores are computed and visited nodes are masked:

$$u_i^m = \frac{(q^m)^T k_i^m}{\sqrt{d_k}}, \quad u_i^m \leftarrow \begin{cases} u_i^m & \text{if } v_i(t) = 0 \\ -\infty & \text{if } v_i(t) = 1 \end{cases} \quad (3.34)$$

Softmax attention weights and weighted aggregation:

$$\alpha_i^m = \frac{\exp(u_i^m)}{\sum_{j: v_j(t)=0} \exp(u_j^m)}, \quad g^m = \sum_{i=0}^n \alpha_i^m v_i^m \in \mathbb{R}^{16} \quad (3.35)$$

3. Head Concatenation and Output Projection. All heads are concatenated and projected:

$$g = [g^1 \parallel g^2 \parallel \dots \parallel g^8] \in \mathbb{R}^{128}, \quad \text{glimpse} = W_{\text{out}} g + b_{\text{out}}, \quad W_{\text{out}} \in \mathbb{R}^{128 \times 128} \quad (3.36)$$

4. Final Pointer Logits. The unnormalised scores are:

$$\ell_i = \frac{\text{glimpse}^T k_i^{\text{logit}}}{\sqrt{128}}, \quad i = 0, \dots, n \quad (3.37)$$

Tanh clipping (prevent numerical overflow) with $\tau_{\text{clip}} = 10$:

$$\ell_i \leftarrow \tau_{\text{clip}} \cdot \tanh\left(\frac{\ell_i}{\tau_{\text{clip}}}\right) \quad (3.38)$$

Final masking of already-visited nodes:

$$\ell_i \leftarrow \begin{cases} \ell_i & \text{if } v_i(t) = 0 \\ -\infty & \text{if } v_i(t) = 1 \end{cases} \quad (3.39)$$

5. Policy. The probability distribution over next nodes:

$$\pi_{\theta}(a_t = i \mid s_t) = \frac{\exp(\ell_i)}{\sum_{j: M_j=0} \exp(\ell_j)} \quad (3.40)$$

Action selection follows Equations 3.7 (training, sampling) and 3.8 (evaluation, greedy). After selecting node a_t , the state is updated and decoding proceeds to step $t + 1$ until all nodes are visited.

3.5 Proposed Architectural Extensions

This section describes the three architectural modifications proposed to improve upon the M1 baseline: the Step-Aware Decoder (Section 3.5), Cost-Aware Gating (Section 3.5), and Time-Sliced Traffic Encoding with Safe Refresh (Section 3.5).

Step-Aware Decoder (Step-MLP)

Standard decoders treat all decoding steps identically, processing each step with the same node embeddings and an updated visited mask, but without explicit awareness of how far the tour has progressed. In the TDTSP, however, decisions made at different stages of the route have fundamentally different characteristics and should be treated accordingly. To address this limitation, we introduce a lightweight Step-MLP that processes step-based features and

injects tour-progress awareness into the decoder query. We choose an MLP because it is the lightest and most computationally efficient option compared to recurrent alternatives such as the GRU or LSTM. We experiment with four complementary mechanisms:

1. **Step feature extraction** (Section 3.5): defines tour-progress signals.
2. **Pre-attention query refinement** (Section 3.5): nonlinear query transformation before attention.
3. **Direct context nudge / Step-MLP** (Section 3.5): linear query offset using explicit features.
4. **Temperature adjustment** (Section 3.5): adaptive exploration control.

Step features. At each decision step t , the decoder constructs a step feature vector f_{step} from configurable tour-progress signals. Table 3.1 defines the available features.

Table 3.1: Step feature definitions for the Step-MLP.

Feature	Dim.	Description	Formula
Step ratio	1	Progress through tour	$r = k/N$
Last-3 nodes	3	Recent visit pattern	$[1_{k-1}, 1_{k-2}, 1_{k-3}]$
Visited mean	128	Spatial tour context	$\bar{h}_{\text{vis}} = \frac{1}{\sum_i v_i} \sum_{i=0}^n v_i h_i$
Unvisited mean	128	Remaining option context	$\bar{h}_{\text{unvis}} = \frac{1}{\sum_i (1-v_i)} \sum_{i=0}^n (1-v_i) h_i$
Sin/Cos time	2	Cyclic time encoding	$[\sin(2\pi t), \cos(2\pi t)]$
Linear time	1	Direct time encoding	$t = \text{state.lengths}$
Depot distance	1	Distance back to depot	$d_{i_t,0}(t)$
Tour length	1	Cumulative travel time	L_t (normalised by $0.1 \cdot N$)
Mean dist to unvisited	1	Average cost to remaining	$\bar{d}_{\text{unvis}} = \frac{1}{\sum_j (1-v_j)} \sum_{j: v_j=0} d_{i_t,j}(t)$

All features are checked for NaN/inf values and replaced with zeros if invalid:

$$f_{\text{step}} \leftarrow \begin{cases} f_{\text{step}} & \text{if finite} \\ 0 & \text{otherwise} \end{cases} \quad (3.41)$$

Five configurable presets are defined for different use cases, as shown in Table 3.2.

Direct Context Nudge (Step-MLP). A three-layer MLP processes the step feature vector $f_{\text{step}} \in \mathbb{R}^{132}$ (default preset v1) to produce a query nudge:

Table 3.2: Step-MLP feature presets.

Preset	Features Included	Dim.	Use Case
Minimal	Step ratio + visited mean	129	Lightweight, fast
v1	Step ratio + last-3 + visited mean	132	Balanced (default)
v2	Step ratio + sin/cos + depot dist + tour length + unvisited mean + mean dist unvisited	134	Full context
v1_light	Step ratio + last-3	4	Minimal compute
v2_light	Step ratio + sin/cos + depot dist + tour length + mean dist unvisited	6	Light, no embeddings

$$h_1 = \text{ReLU}(W_1 f_{\text{step}} + b_1), \quad W_1 \in \mathbb{R}^{64 \times 132}, \quad h_1 \in \mathbb{R}^{64} \quad (3.42)$$

$$h_2 = \text{ReLU}(W_2 h_1 + b_2), \quad W_2 \in \mathbb{R}^{64 \times 64}, \quad h_2 \in \mathbb{R}^{64} \quad (3.43)$$

$$\Delta q = W_3 h_2 + b_3, \quad W_3 \in \mathbb{R}^{128 \times 64}, \quad \Delta q \in \mathbb{R}^{128} \quad (3.44)$$

The context nudge Δq is added to the base query:

$$q_{\text{final}} = q_{\text{base}} + \Delta q \quad (3.45)$$

Pre-Attention Query Refinement. A two-layer feed-forward network nonlinearly refines the query q_t before the attention computation:

$$\text{refined_query} = W_2 \sigma(W_1 \cdot q_t + b_1) + b_2 \quad (3.46)$$

The refined output is added back as a residual:

$$q_{\text{new}} = q_{\text{old}} + \text{refined_query} \quad (3.47)$$

Temperature Adjustment (Temp-MLP). This mechanism improves accuracy by scaling the attention logits with a learned temperature factor. A lightweight MLP computes:

$$h = \text{ReLU}(W_1 f_{\text{step}} + b_1), \quad W_1 \in \mathbb{R}^{64 \times 132}, \quad h \in \mathbb{R}^{64} \quad (3.48)$$

$$\tau_{\text{raw}} = \text{Sigmoid}(W_2 h + b_2), \quad W_2 \in \mathbb{R}^{1 \times 64}, \quad \tau_{\text{raw}} \in [0, 1] \quad (3.49)$$

The raw temperature is scaled to a practical range:

$$\tau_{\text{step}} = 0.5 + 2.0 \cdot \tau_{\text{raw}} \in [0.5, 2.5] \quad (3.50)$$

Values $\tau_{\text{step}} < 1.0$ make the distribution more greedy (peaked), while $\tau_{\text{step}} > 1.0$ increases entropy (more exploration); $\tau_{\text{step}} = 1.0$ is the baseline behaviour. The effective temperature is $T_{\text{eff}} = T \cdot \tau_{\text{step}}$ (global hyperparameter T , default 1), and log-probabilities are computed as:

$$\log(p_j) = \log \left(\text{softmax} \left(\frac{\ell}{T \cdot \tau_{\text{step}}} \right)_j \right) \quad (3.51)$$

Post-Attention Glimpse Refinement. In the baseline decoder, only a single linear projection is used after the MHA for the glimpse. We propose a post-attention MLP that refines the glimpse $g_t \in \mathbb{R}^{128}$ after the MHA but before the final logit computation:

$$h = \text{ReLU}(W_1 g_t + b_1), \quad W_1 \in \mathbb{R}^{512 \times 128} \quad (3.52)$$

$$g_{\text{refined}} = W_2 h + b_2, \quad W_2 \in \mathbb{R}^{128 \times 512} \quad (3.53)$$

The refined glimpse replaces the original (no residual connection): $g_{\text{final}} = g_{\text{refined}}$. Final logits are then:

$$\ell_i = \frac{g_{\text{final}}^T k_i^{\text{logit}}}{\sqrt{d}} \quad (3.54)$$

Cost-Aware Gating and Heuristic Blending

Classical routing heuristics (nearest neighbour, greedy cost minimisation) often provide reasonable solutions with interpretable and computationally inexpensive scores. We inject this classical knowledge into the decoder by adding a soft bias to the attention logits.

Heuristic Logit Computation

Three classical routing heuristics are implemented, each providing a score $h_i(s_t) \in \mathbb{R}$ for candidate node i at step t .

1. Linear Travel Time (Nearest Neighbour) Heuristic. This heuristic selects nodes with the shortest travel time from the current node:

$$h_i^{\text{lin}} = -d_{i_t, i}(t) \quad (3.55)$$

2. NN+1 Lookahead Heuristic. This strategy selects nodes based not only on the immediate cost but also on how good the next move will be. For each candidate next node i :

- Immediate cost: $c_i = d_{i_t, i}(t)$
- Lookahead cost: $c_i^{\text{next}} = \min_{j \in U_t \setminus \{i\}} d_{i, j}(t + c_i)$ (best follow-up cost from i)

$$\text{score}_i = c_i + \lambda_{\text{lookahead}} \cdot c_i^{\text{next}}, \quad h_i = -\text{score}_i, \quad \lambda_{\text{lookahead}} = 0.5 \quad (3.56)$$

3. Nearest Neighbour Random (NNR) Heuristic. NNR assigns probability-based scores to the top- k ($k = 3$) candidate nodes. Candidates are sorted by travel time (i_1 has shortest) and assigned probabilities $p_1 = 0.80$, $p_2 = 0.15$, $p_3 = 0.05$:

$$h_i^{\text{NNR}} = \begin{cases} \log(0.80) \approx -0.22 & \text{if } i = i_1 \\ \log(0.15) \approx -1.90 & \text{if } i = i_2 \\ \log(0.05) \approx -3.00 & \text{if } i = i_3 \\ -10.0 & \text{otherwise} \end{cases} \quad (3.57)$$

Heuristic Blending

The heuristic scores are blended with the attention logits ℓ_i from Stage 3 (Section 3.4) via a linear gating mechanism:

$$\ell_i^{\text{blended}} = \ell_i + \lambda_{\text{heur}} \cdot h_i(s_t) \quad (3.58)$$

where $\lambda_{\text{heur}} \in [0, 2.0]$ is a learnable parameter, initialised to $\lambda_{\text{heur}} = 1.0$. Optionally, the heuristic logits can be transformed nonlinearly before blending, either using a **Piecewise Linear Transform** ($h_i^{\text{transformed}} = \text{MLP}_{\text{transform}}(h_i)$) or an **Exponential Transform**:

$$h_i^{\text{transformed}} = \alpha \cdot \exp(h_i + \beta) \quad (3.59)$$

where α, β are learnable parameters.

Time-Sliced Traffic Encoding and Safe Refresh

The baseline M1 model feeds the full-day traffic profile for every arc (i, j) into the encoder. In real-world dynamic routing (e.g., logistics or food delivery), operations have a specific opening and closing time; feeding the full-day profile wastes computation and introduces noise from irrelevant time periods. By feeding only a forward time window from the current decision time, model complexity is reduced while accuracy is maintained.

Figure 3.3 illustrates the concept of time slicing, where only the forward time bins starting from the current time are selected.

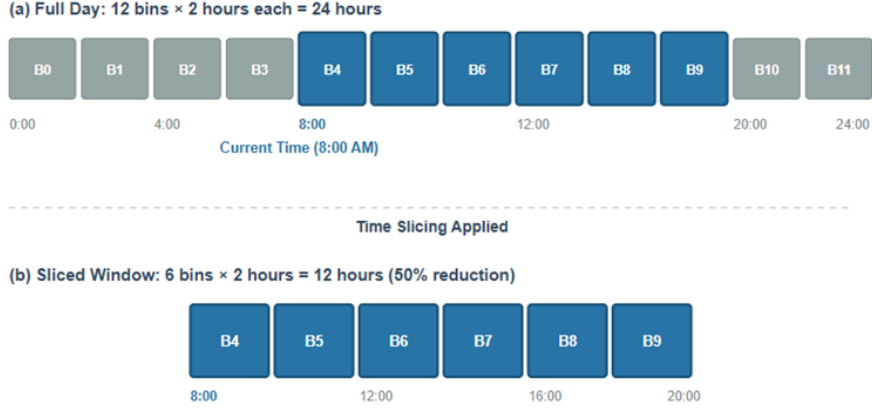


Figure 3.3: Time slicing applied to the traffic profile. (a) Full-day representation with 12 bins of 2 hours each. (b) Sliced window of 6 bins (50% reduction) starting from the current time (8:00 AM).

Our approach has three main components: (1) **slicing** the traffic tensor to a relevant forward window, (2) **embedding** this window instead of the full day, and (3) **monitoring** tour progress and **selectively re-encoding** when the window becomes stale.

Windowed Traffic Representation

The traffic data is stored as cubic spline constant coefficients in a flat array of length $n_c \cdot n_c \cdot T = 100 \times 100 \times 12 = 120,000$. The flat index for source i , destination j , bin b is: $\text{idx}(i, j, b) = i \cdot n_c \cdot T + j \cdot T + b$.

With time slicing, the matrix is reshaped into (n_c, n_c, T) and reduced to a window of size W bins starting at `start_bin` (defined by the user as either a normalised time $\tau \in [0, 1]$ or a bin index). Two cases arise when extracting the window (`end_bin = start_bin + W`):

- **Normal case** (`end_bin ≤ T`):

$$\text{mat_windowed} = \text{mat_reshaped}[:, :, \text{start_bin} : \text{end_bin}] \in \mathbb{R}^{n_c \times n_c \times W}$$

- **Wraparound case** (`end_bin > T`): the window crosses midnight. Two parts are concatenated:

$$\begin{aligned} \text{part}_1 &= \text{mat_reshaped}[:, :, \text{start_bin} : T] \\ \text{part}_2 &= \text{mat_reshaped}[:, :, 0 : (\text{end_bin} - T)] \\ \text{mat_windowed} &= \text{cat}(\text{part}_1, \text{part}_2, \text{dim} = 2) \in \mathbb{R}^{n_c \times n_c \times W} \end{aligned}$$

The last two dimensions are flattened to give the windowed traffic tensor $z_{\text{windowed}} \in \mathbb{R}^{1 \times n_c \times (n_c \cdot W)}$, where each node has $n_c \cdot W$ features instead of 1,200. Values outside the selected window are padded with zeros to maintain the fixed input size of the existing traffic embedding layer.

Safe Refresh Mechanism

Since the encoder uses time-sliced data, it must be refreshed when the current decision time approaches or exceeds the encoded window. Four strategies are implemented:

Strategy 1: One-Time Refresh Check. If the decoder’s current bin exceeds the window end bin, the encoder re-runs with the latest values before the next decoding step, setting the new start bin to the current bin.

Strategy 2: Periodic Refresh. A refresh is triggered on a fixed schedule regardless of window boundaries. At each decoding step, it computes $\text{time_since_refresh} = \text{current_time} - \text{last_refresh_time}$ and triggers a refresh if $\text{time_since_refresh} \geq \text{refresh_interval}$. This mitigates the risk of using stale embeddings.

Strategy 3: k -Moves Lookahead. The encoder is refreshed before the boundary is reached by simulating k moves ahead. The future bin is estimated either by adding k time bins to the current time, or by simulating the next k moves via greedy decoding or nearest-neighbour methods. In this thesis, we adopt the approach of adding k time bins to the current time, as it is simpler and computationally more efficient than simulating future moves. A refresh is triggered if the estimated future bin $\geq$ the current window end bin.

Strategy 4: Combined. A combination of all three strategies; the encoder is refreshed whenever any of their criteria is satisfied.

3.6 Training Methodology

This section describes the data generation process, the REINFORCE training algorithm, and the baseline strategies used to train the attention-based decoder.

Data Generation and Preprocessing

Problem Instance Generation. The model operates on a fixed set of 100 cities with 2D coordinates uniformly distributed from $[0, 1]^2$, though this can be extended to more than 100 cities if required. These coordinates are not

used directly in the M1 model, as the TDTSP requires travel times rather than raw coordinates, and travel times are derived from the traffic data. Each training instance uses a random subset of size `graph_size + 1` (including the depot), selected by generating random scores for each city, sorting in ascending order, and taking the first n cities plus the depot. Each instance is represented as a one-hot tensor of shape $(\text{graph_size} + 1, n_{\text{cities}})$, where each row indicates the selected position in the 100-city pool. Training generates 128,000 instances per epoch in batches of 512. A fixed validation set of 1,000 instances is used to report validation performance. The same fixed test set is used across all random seeds.

Travel-Time Data and Preprocessing. Following the baseline authors, we use the real-world Beijing (Jingdong Logistics) dataset, which consists of 100 customers (including the depot) with 10,000 order-pair rows (one row per directed edge (i, j)).

Time Discretisation. The 24-hour day is divided into $T = 12$ time bins of 2 hours each (00:00–02:00, 02:00–04:00, $\dots$, 22:00–24:00). The dataset provides one travel time measurement per bin for each edge, yielding 12 values per row.

Cubic Spline Interpolation. To enable smooth travel time computation at any departure time (not just at bin boundaries), cubic spline interpolation is applied. For each edge (i, j) , a periodic cubic spline is fitted to the 12 discrete measurements. This produces four coefficients $(c_{ij,b}^0, c_{ij,b}^1, c_{ij,b}^2, c_{ij,b}^3)$ per bin b . Exactly four coefficients arise because of the periodic C^2 -smoothness conditions. These conditions require values, first derivatives, and second derivatives to match at all 12 bin boundaries, including the wrap from bin 11 back to bin 0. Together, they form a 48×48 linear system that uniquely determines 4 coefficients for each of the 12 bins. The coefficients are stored in `mat.mat` ($c_{ij,b}^0$), `mat.m2` ($c_{ij,b}^1$), `mat.m3` ($c_{ij,b}^2$), and `mat.m4` ($c_{ij,b}^3$). This yields $100 \times 100 \times 12 = 120,000$ coefficient sets in total. The preprocessing steps are:

1. Load periodic time intervals from 0 to 12 and corresponding travel times as inputs (with the first value repeated for periodicity).
2. Fit the cubic spline and extract 4 coefficients per bin per edge: $(c_{ij,b}^0, c_{ij,b}^1, c_{ij,b}^2, c_{ij,b}^3)$.
3. Store coefficients in four separate flat arrays (`mat.mat` ($= c_{ij,b}^0$), `mat.m2` ($= c_{ij,b}^1$), `mat.m3` ($= c_{ij,b}^2$), `mat.m4` ($= c_{ij,b}^3$)) of size $N_{\text{total}} \times N_{\text{total}} \times T = 100 \times 100 \times 12 = 120,000$ values each.

Travel Time Computation. For edge (i, j) at any departure time $t \in [0, 1)$ (normalised to a 24-hour day), the travel time is computed as:

$$d_{ij}(t) = c_{ij,b}^0 + c_{ij,b}^1 z + c_{ij,b}^2 z^2 + c_{ij,b}^3 z^3 \quad (3.60)$$

where:

- $b = \lfloor t \cdot T \rfloor$ is the time bin index (0 to 11)
- $z = (t \cdot T - \lfloor t \cdot T \rfloor) / T \in [0, 1/T)$ is the fractional position within the bin, normalised by T
- $(c_{ij,b}^0, c_{ij,b}^1, c_{ij,b}^2, c_{ij,b}^3)$ are the four coefficients specific to edge (i, j) at time bin b

For example, if $t = 0.375$ (representing 9:00 AM), then $b = \lfloor 0.375 \times 12 \rfloor = 4$ (the 5th bin, covering 8:00–10:00 AM), and $z = 0.375 \times 12 - 4 = 0.5$, indicating the midpoint of that bin.

Stability Clamp. To prevent numerical instability from extreme values, computed travel times are clamped to a reasonable range (see Equation 3.26). Total storage is $100 \times 100 \times 12 = 120,000$ coefficients per array.

Encoder vs Decoder Usage. The encoder uses only the constant coefficient (c_0) as the base travel time feature for each edge, representing the average travel time for that bin. The decoder uses all four coefficients to compute precise interpolated travel times at the exact departure time via Equation 3.23.

Three independent datasets are used throughout training, as summarised in Table 3.3.

Table 3.3: Datasets used in training and evaluation.

Dataset	Size (Default)	Sampling	Purpose
Training	128,000	Random per epoch	Reduces overfitting
Validation/Test	1,000	Fixed	Reports validation performance; final evaluation
Baseline Validation	1,000	Fixed	Statistical testing for baseline updates

Policy Gradient: REINFORCE Algorithm

Since the TDTSP has no labelled tour data, reinforcement learning is used to train the decoder. The objective is to learn a policy π_θ that minimises the expected tour cost:

$$J(\theta) = \mathbb{E}_{s \sim \mathcal{D}_{\text{train}}} [C(\pi_\theta(s))] \quad (3.61)$$

For each instance, the model outputs a tour cost C and the log-likelihood of the sampled tour. The tour cost is:

$$C_k = \sum_{(i,j) \in \text{tour}} d_{ij}(t_{\text{arrival at } i}) \quad (3.62)$$

The log-likelihood of the tour is:

$$\log p_\theta(\pi \mid x) = \sum_t \log p_\theta(a_t \mid s, a_{<t}) \quad (3.63)$$

Using the REINFORCE policy gradient estimator, the gradient of $J(\theta)$ is approximated by:

$$\nabla_\theta J(\theta) \approx \mathbb{E}[(C(\pi_\theta(s)) - b(s)) \nabla_\theta \log p_\theta(\pi_\theta(s) \mid s)] \quad (3.64)$$

where $b(s)$ is the baseline cost. For a mini-batch of size B , the batch-wise REINFORCE loss is:

$$L_{\text{REINFORCE}} = \frac{1}{B} \sum_{i=1}^B (C_i - b_i) \log p_\theta(\pi_i \mid x_i) \quad (3.65)$$

After computing the loss, backpropagation is performed, followed by ℓ_2 -norm gradient clipping with threshold τ :

$$g = \nabla_\theta L_{\text{REINFORCE}}, \quad \tilde{g} = \text{clip}_{\ell_2}(g, \tau) = \frac{g}{\max(1, \|g\|_2/\tau)} \quad (3.66)$$

Parameters are then updated using the Adam optimiser with learning rate α :

$$\theta \leftarrow \theta - \alpha \tilde{g} \quad (3.67)$$

Baseline Strategies

Three baseline strategies are implemented, the user can choose any option according to their requirements, each with different trade-offs.

Exponential Moving Average Baseline. The EMA baseline maintains a moving average of batch costs, exponentially weighted towards recent batches. If a batch b consists of B instances with batch cost $\tilde{C}_b = \frac{1}{B} \sum_{k=1}^B C_{b,k}$, the baseline is updated as:

$$v_b = \begin{cases} \tilde{C}_0 & \text{if } b = 0 \\ \beta \cdot v_{b-1} + (1 - \beta) \cdot \tilde{C}_b & \text{if } b > 0 \end{cases} \quad (3.68)$$

where $\beta \in [0, 1]$ is the scaling factor. The EMA is a scalar value (not a model) and is updated at every batch.

Rollout Baseline. The rollout baseline uses a separate frozen copy of a previously defined model to provide instance-level baseline costs. In this process, two models are maintained, namely the training model and the baseline model. At the start of training, the baseline model is initialised as a copy of the training model and runs in greedy mode on the baseline validation dataset to produce costs. At the end of each epoch, the current model's performance on the baseline validation dataset is compared against the baseline model's costs. The per-instance differences $d_i = C_{\text{current},i} - C_{\text{baseline},i}$ are computed and a one-sided paired t -test is applied:

$$t = \frac{\bar{d}}{\text{SE}(d)}, \quad \text{p-value} = P(T < t) \quad (3.69)$$

where $\bar{d}$ is the mean difference and $\text{SE}(d)$ is the standard error of the differences. If $\tilde{C}_{\text{current}} < \tilde{C}_{\text{baseline}}$ and $\text{p-value} < 0.05$, the current model is statistically significantly better and the baseline model is updated. Otherwise, the baseline model is kept.

Warmup Baseline. In the early epochs, the rollout baseline produces noisy predictions because the weights are randomly initialised and not yet sufficiently trained. The Warmup baseline provides EMA during early epochs and gradually transitions to rollout as the model improves. Let K_{WU} be the number of warmup epochs. At epoch e , the blend factor is:

$$\gamma(e) = \frac{\min(e + 1, K_{\text{WU}})}{K_{\text{WU}}} \quad (3.70)$$

The blended baseline is:

$$b_i^{\text{blended}} = \gamma(e) \cdot b_i^{\text{rollout}} + (1 - \gamma(e)) \cdot b_i^{\text{EMA}} \quad (3.71)$$

Training Process and Baseline Update

Figure 3.4 illustrates the complete training workflow, showing how training instances flow through batch processing and how baselines are updated.

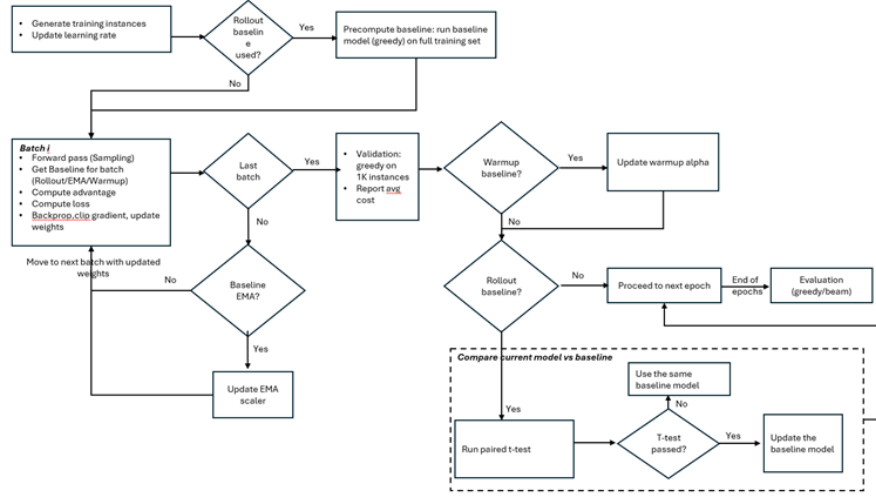


Figure 3.4: Complete training workflow for the attention-based decoder with REINFORCE and baseline strategies. The dashed box indicates the baseline update logic applied at the end of each epoch.

Each training epoch follows this structure:

1. The learning rate is decayed by multiplying with a scalar (default: 0.995).
2. A new training dataset is generated by random sampling (default: 128,000 instances) to reduce overfitting.
3. If using the rollout baseline, the frozen baseline model performs greedy decoding on the entire training dataset at the start of the epoch. The resulting costs b_i are stored and paired with each training instance throughout the epoch.
4. The dataset is divided into mini-batches (default batch size: 512). Each batch follows the per-batch training step described below.
5. After all batches, the current model is evaluated on the fixed validation dataset (1,000 instances) using greedy decoding, and the mean validation cost is reported.
6. If using the warmup baseline, the blend factor $\gamma(e)$ is updated (Section 3.6).
7. If using the rollout baseline, the baseline model is compared with the current model via a paired t -test on the baseline validation dataset

(Section 3.6). The baseline is updated only if the current model shows statistically significant improvement.

8. Model state, optimiser state, and baseline state are saved at specified intervals.

Per-Batch Training Step. For each mini-batch, the training model runs in stochastic sampling mode, producing a cost vector C (one cost per instance) and the log-likelihood $\log p_\theta(\pi \mid x)$ of each sampled tour. The baseline value is computed according to the selected strategy (rollout uses precomputed epoch-start values; EMA computes a running average; warmup blends both). The REINFORCE loss is computed (Equation 3.65), followed by backpropagation, gradient clipping (Equation 3.66), and parameter update (Equation 3.67). After training completes, the best performing model is evaluated on the test dataset using greedy decoding.

3.7 Graph-Size-Independent Architecture

The baseline M1 model (Zhang et al., 2023) was originally designed and evaluated for fixed customer set sizes of $C = 4, 9$, and 19 , sampled from a pool of 100 customers. As a result, the encoder-decoder architecture is dependent on the graph size at training time, meaning the model must be retrained from scratch whenever the customer set size changes. This dependency is challenging in real-world applications, as the customer base varies from time to time and therefore a more generalised model is needed.

The graph size dependency mainly impacts the dynamic input embedding stage, which is the part of the decoder that builds the stepwise context vector (Section 3.4). When building the context vector, five components are required; among these, the *visited state projection* and the *current traffic projection* are graph-size-dependent. This section describes how these two components are replaced to produce a fully graph-size-independent architecture.

Visited State Projection

In the original model, the visited state vector $v(t)$ has a fixed length of $n + 1$ (number of nodes including depot). For graph size $n = 19$, this gives:

$$v(t) \in \mathbb{R}^{20}, \quad W_v \in \mathbb{R}^{128 \times 20} \quad (3.72)$$

The linear projection layer performs the matrix multiplication $(128 \times 20) \cdot (20 \times 1) = (128 \times 1)$, which works correctly during training. However, if the same model is used for graph size $n = 29$:

$$v(t) \in \mathbb{R}^{30}, \quad W_v \in \mathbb{R}^{128 \times 20} \quad (3.73)$$

The multiplication $(128 \times 20) \cdot (30 \times 1)$ is **impossible** as the dimensions do not match and the model crashes. A graph size of 29 would therefore require retraining the entire model, as the linear projection layer needs 128×30 weights.

To mitigate this issue, we introduce an adaptive average pooling layer before the linear projection:

$$\tilde{v}(t) = W_v \cdot \text{AdaptiveAvgPool1d}(128)(v(t)) \in \mathbb{R}^{128}, \quad W_v \in \mathbb{R}^{128 \times 128} \quad (3.74)$$

`AdaptiveAvgPool1d(128)` maps any input of arbitrary length to exactly 128 values. When the input is **smaller** than 128 (as in the visited state case), it expands by repeating values. The mapping from output index to input position is:

$$\text{input_position} = \frac{\text{output_index} \times \text{input_size}}{\text{output_size}} \quad (3.75)$$

For graph size = 19, $v(t) \in \mathbb{R}^{20}$, output size = 128. Each input value covers $128/20 \approx 6.4$ output positions:

- Input index 0 (visited) $\rightarrow$ repeated across output positions 0–6 (7 slots)
- Input index 1 (unvisited) $\rightarrow$ repeated across output positions 7–12 (6 slots)
- Input index 2 (unvisited) $\rightarrow$ repeated across output positions 13–19 (7 slots)
- ... and so on for all 20 input values.

For graph size = 29, $v(t) \in \mathbb{R}^{30}$, output size = 128. Each input value covers $128/30 \approx 4.3$ output positions:

- Input index 0 $\rightarrow$ repeated across output positions 0–4 (5 slots)
- Input index 1 $\rightarrow$ repeated across output positions 5–8 (4 slots)
- ... and so on for all 30 input values.

After adaptive pooling, the embeddings pass through the linear projection layer where $W_v \in \mathbb{R}^{128 \times 128}$ remains unchanged regardless of the original graph size. Inside the linear projection layer, $(128 \times 128) \cdot (128 \times 1) = (128 \times 1)$, which is compatible with the context vector for any graph size.

Traffic Embedding Projection

The current traffic embedding is more complex. At each decoding step, the decoder computes travel times across **all pairwise combinations** of nodes, not just from the current node. This requires first generating all pairs, then looking up their travel times, and finally projecting to 128 dimensions. Three steps are involved.

Step 1: Generating Pairwise Indices (`_generate_xx_yy`).

For **graph size** = 19 (20 nodes, indices 0–19), all pairwise combinations are:

$$\mathbf{xx} = [\underbrace{0, \dots, 0}_{20}, \underbrace{1, \dots, 1}_{20}, \dots, \underbrace{19, \dots, 19}_{20}] \in \mathbb{R}^{400} \quad (3.76)$$

$$\mathbf{yy} = [\underbrace{0, \dots, 19}_{20}, \underbrace{0, \dots, 19}_{20}, \dots, \underbrace{0, \dots, 19}_{20}] \in \mathbb{R}^{400} \quad (3.77)$$

Total pairs = $20 \times 20 = 400$. For **graph size** = 29 (30 nodes): $\mathbf{xx} \in \mathbb{R}^{900}$, $\mathbf{yy} \in \mathbb{R}^{900}$, total pairs = $30 \times 30 = 900$.

In the original implementation, `self.xx` and `self.yy` were pre-computed at model initialisation with a fixed size. A model trained on graph size 19 (shape 1×400) would use incorrect indices when run on graph size 29, producing model crashes. We replace this with the dynamic function `_generate_xx_yy`, which computes the correct pairwise indices at runtime based on the actual graph size of the current input.

Step 2: Looking Up Travel Times.

Using $\mathbf{xx}$ and $\mathbf{yy}$, the distance matrix returns the travel time for every pair at the current time t via cubic spline interpolation (Section 3.2):

$$w(t) = [d_{0,0}(t), d_{0,1}(t), \dots, d_{0,n}(t), d_{1,0}(t), \dots, d_{n,n}(t)] \in \mathbb{R}^{(n+1)^2} \quad (3.78)$$

For graph size 19: $w(t) \in \mathbb{R}^{400}$. For graph size 29: $w(t) \in \mathbb{R}^{900}$.

Step 3: Adaptive Pooling Projection.

In the original model:

$$\tilde{w}(t) = W_w \cdot w(t), \quad W_w \in \mathbb{R}^{128 \times 400} \quad (3.79)$$

For graph size 29, $w(t) \in \mathbb{R}^{900}$, so the weight matrix $W_w \in \mathbb{R}^{128 \times 400}$ cannot accept 900 inputs. After applying adaptive pooling:

$$\tilde{w}(t) = W_w \cdot \text{AdaptiveAvgPool1d}(128)(w(t)) \in \mathbb{R}^{128}, \quad W_w \in \mathbb{R}^{128 \times 128} \quad (3.80)$$

This time the input is **larger** than 128, so the pooling **compresses** by averaging. The start and end positions for each output segment are:

$$\text{start} = \left\lfloor \frac{\text{output_index} \times \text{input_size}}{\text{output_size}} \right\rfloor, \quad \text{end} = \left\lfloor \frac{(\text{output_index} + 1) \times \text{input_size}}{\text{output_size}} \right\rfloor \quad (3.81)$$

For **graph size = 19**, $w(t) \in \mathbb{R}^{400}$, output size = 128. Each output covers $400/128 \approx 3.1$ input values. Table 3.4 illustrates the pooling for example values $w(t) = [0.5, 0.3, 0.8, 0.2, 0.6, 0.4, 0.9, 0.1, 0.7, \dots]$.

Table 3.4: Adaptive average pooling example for graph size = 19 ($\mathbb{R}^{400} \rightarrow \mathbb{R}^{128}$).

Output index	Input indices	Values	Average
0	[0, 1, 2]	[0.5, 0.3, 0.8]	0.533
1	[3, 4, 5]	[0.2, 0.6, 0.4]	0.400
2	[6, 7, 8]	[0.9, 0.1, 0.7]	0.567
...	...	...	...

For **graph size = 29**, $w(t) \in \mathbb{R}^{900}$, output size = 128. Each output covers $900/128 \approx 7.0$ input values. Table 3.5 shows the pooling mapping.

Table 3.5: Adaptive average pooling example for graph size = 29 ($\mathbb{R}^{900} \rightarrow \mathbb{R}^{128}$).

Output index	Input indices	Values averaged
0	[0, 1, 2, 3, 4, 5, 6]	average of $d_{0,0}(t)$ to $d_{0,6}(t)$
1	[7, 8, 9, 10, 11, 12, 13]	average of $d_{0,7}(t)$ to $d_{0,13}(t)$
...	...	...

Importantly, all previously described modifications are fully compatible with the graph-size-independent architecture. These include Step-MLP, Decoder MLP variants, Cost-Aware Gating, and Time Slicing with Safe Refresh. Both the graph-dependent and graph-independent versions of the full model are evaluated experimentally. The graph-independent version is additionally tested by training on one graph size and evaluating on a different graph size, without any retraining. These experiments are described in Section 4.8.

3.8 Summary

This chapter presented the complete methodology for improving the M1 baseline on the deterministic TDTSP with a fixed customer set. Starting from the formal MDP formulation, the encoder-decoder architecture was described in detail, covering the graph attention encoder, the three-stage attention-based pointer decoder, and its adaptation to the deterministic setting. Three architectural extensions were proposed. First, the Step-Aware Decoder (Step-MLP) and decoder variants introduce tour-progress awareness into the decoder query. Second, Cost-Aware Gating and Heuristic Blending inject classical routing knowledge into the learned logits. Third, Time-Sliced Traffic Encoding with Safe Refresh reduces encoder complexity by focusing on a relevant forward time window. Additionally, a graph-size-independent architecture was introduced, replacing the fixed-size visited state and traffic embedding projections with adaptive average pooling layers, enabling the trained model to generalise across different customer set sizes without retraining. The training methodology employs the REINFORCE algorithm with three baseline strategies (EMA, Rollout, Warmup), trained on cubic spline-interpolated real-world Beijing traffic data. The experimental evaluation of these extensions is presented in the following chapter.

Chapter 4

Experiments

All experiments were executed on the university HPC cluster (STUD partition). Each experiment was allocated 4 CPU cores and a single NVIDIA GeForce RTX 2070 GPU with 8 GB of memory.

4.1 Experimental Setup

Problem and Dataset. All methods are evaluated using a real-world dataset from Beijing, which was also used by the authors of the baseline paper (Zhang et al., 2023). The dataset consists of 100 locations (including the depot), and travel times are recorded for every ordered pair (i, j) , yielding 10,000 city pairs in total. The 24-hour day is divided into 2-hour intervals (giving 12 time bins per arc), with each interval storing a normalised travel time. Rather than using raw (actual) values, the travel times are preprocessed using cubic spline interpolation. Full details of this procedure are described in Section 3.6. Out of these 100 locations, this study focuses on four customer sets (C):

- $C = 19$: used for comparisons in the experiments.
- $C = 20$: used for hyperparameter optimisation.
- $C = 29$: intermediate scalability check and graph dependency experiments.
- $C = 49$: used for larger-scale and scalability experiments.

Each training instance uses a random subset of the 100 cities of size `graph_size + 1` (including the depot), selected by random scoring and sorting as described in Section 3.6. Nodes are represented as one-hot encoded vectors

of dimension 100 rather than raw coordinates, since the model operates on time-dependent travel times rather than Euclidean distances.

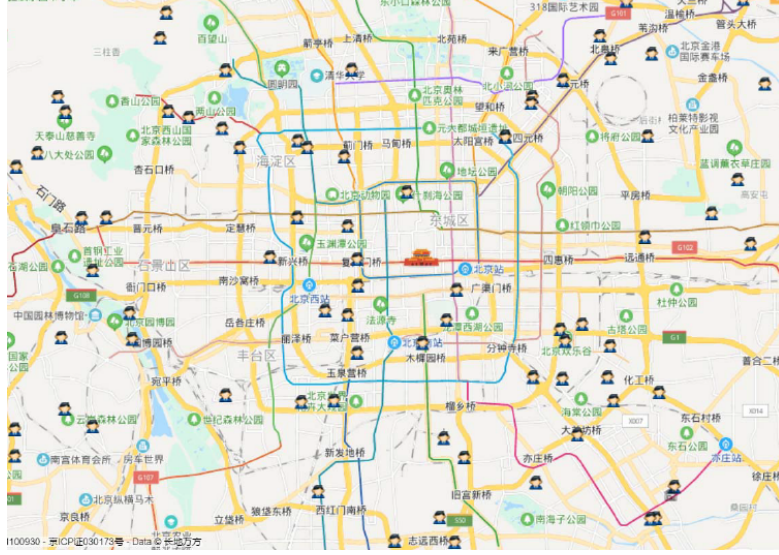


Figure 4.1: Location map of the benchmark dataset (Beijing). The depot is located at the bottom-right of the map. Reproduced from (Zhang et al., 2023).

Models and variants. As discussed in Section 3.5, this chapter evaluates:

- Four different step-aware decoder configurations with different combinations of features.
- Three different heuristic model blending configurations using cost-aware gating.
- Three different time-sliced window encoding and safe refreshing mechanism configurations.

Heuristic Baselines. The proposed model’s performance is compared with the following classical heuristics: Nearest Neighbour (NN), Greedy Edge, Randomised Nearest Neighbour (NNR), look-ahead-1 (NN+1), and 2-opt initialised with Nearest Neighbour.

For a fair comparison, all heuristics are evaluated on the same problem instances and use the same cubic spline interpolation method.

Training. All models are trained with policy gradient reinforcement learning with a rollout baseline using the following hyperparameters:

Table 4.1: Hyperparameters explored and selected values.

Hyperparameter	Values Explored	Selected Value
Batch size	32, 256, 512	256 (see Section 4.4)
Epochs	50, 100	50 ($C = 19/20$), 100 ($C = 29/49$)
Learning rate	0.00005, 0.0001, 0.00015, 0.0002	0.0001 (see Section 4.5)
LR decay	0.99, 0.995, 0.998	0.995 (see Section 4.5)
Step-MLP hidden dim	32, 64, 128	64 (see Section 4.5)
Random seeds	1234, 5678, 9012	All three (averaged)

To ensure robust results, three predefined random seeds are used, and all reported numbers are averages of the three random seeds. Sampling is used for training, and greedy decoding is used for both the baseline model and evaluation, unless otherwise specified. The user may optionally select beam search for evaluation, and those results are reported separately in Section 4.9.

4.2 Baseline Model (M1) Performance

Before evaluating the proposed modifications, the performance of the M1 baseline is first established as an unmodified reference point. This baseline uses the full-day traffic profile, without decoder MLP variants or cost-aware gating. All results use greedy decoding and are reported as mean test cost in minutes.

Effect of Training Duration ($C = 19$). Using the architecture described in Chapter 3, the baseline model (M1) is trained for $C = 19$ nodes with 50 and 100 epochs. Table 4.2 shows that training for 100 epochs improves performance by approximately 9.2 minutes (574.73 vs 565.50 min), with significantly reduced variance (10.78 vs 0.58). Based on this finding, all experiments on larger graph sizes ($C = 29$, $C = 49$) use 100 training epochs as the default, while experiments focused on feature selection and hyperparameter search at $C = 19$ and $C = 20$ use 50 epochs to reduce computational cost and obtain results more quickly.

Table 4.2: Baseline M1 performance across training epochs and graph sizes.

Graph Size	Epochs	Mean $\pm$ SD (min)
$C = 19$	50	574.73 $\pm$ 10.78
$C = 19$	100	565.50 $\pm$ 0.58
$C = 29$	100	766.79 $\pm$ 7.39
$C = 49$	100	1095.93 $\pm$ 73.34

These baseline results serve as the comparison target throughout subsequent sections.

4.3 Step-MLP Variants

This section evaluates the proposed Step-MLP and related decoder enhancements. All experiments use $C = 19$ with 50 epochs unless otherwise stated, with the reference baseline of 574.73 ± 10.78 minutes (Table 4.2). The Step-MLP adds a context nudge to the decoder using step-aware features passed through a three-layer MLP (Section 3.5).

Feature Selection for Step-MLP. Individual features (step ratio, linear time, depot distance, tour length, mean dist unvisited, last 3 visited nodes) were first evaluated independently. No single feature outperformed the baseline, ranging from 577.9 to 581.5 minutes, compared to 574.7 minutes for the baseline, though linear time and mean distance to unvisited nodes performed best among them.

Several predefined feature combinations were then evaluated (Table 4.3). The `v2_light` preset with linear time encoding achieves the lowest mean cost (573.80 min). However, this difference of 0.93 minutes lies well within one standard deviation of the baseline (574.73 ± 10.78 min) and cannot be considered a statistically significant improvement at this scale. Notably, `v2_light` exhibits substantially reduced variance (± 7.22 vs. ± 10.78), indicating more stable training across seeds. Higher-dimensional feature sets (129 to 132 dimensions) do not offer consistent improvements, suggesting embedding-based features introduce noise at this problem size.

Table 4.3: Feature preset comparison ($C = 19$, 50 epochs).

Feature Set	Baseline	v1 light (4)	minimal (129)	v1 (132)	v2 light (linear, 5)
Mean $\pm$ SD (min)	574.73 ± 10.78	580.00 ± 3.42	575.60 ± 8.00	574.95 ± 7.36	573.80 ± 7.22

A leave-one-out ablation on `v2_light`, which has five features, (Table 4.4) reveals that linear time is the most critical feature, as removing it increases cost by 7.5 minutes. Replacing linear time with sinusoidal encoding also degrades performance (578.2 vs. 573.8 min), likely because travel time bins represent a single non-repeating operational day. Based on its stable training behaviour and lowest mean cost among all tested presets, `v2_light` with linear time encoding is selected for all subsequent experiments.

Table 4.4: Leave-one-out ablation of v2_light ($C = 19$, 50 epochs).

Removed Feature	Mean Cost (min)	Δ vs. Full v2_light
None (full v2_light)	573.80	-
linear_time	581.34	+7.49
mean_dist_unvisited	578.80	+4.95
step_ratio	577.96	+4.11
tour_length	577.12	+3.27
depot_distance	574.55	+0.70

Temperature MLP and Decoder MLP Variants. Beyond the Step-MLP context nudge, three additional mechanisms were evaluated: Temperature MLP (scales attention logits), Option C (pre-attention MLP), and Option A (post-attention MLP). Table 4.5 summarises results at $C = 19$.

Table 4.5: Decoder MLP variant results ($C = 19$, 50 epochs).

Configuration	Mean $\pm$ SD (min)
Baseline	574.73 $\pm$ 10.78
Temp-MLP standalone	568.68 $\pm$ 2.12
Option C only	570.39 $\pm$ 1.85
Option A only	570.32 $\pm$ 2.10
Step-MLP + Temp-MLP	576.00 $\pm$ 5.59
Option C + Temp-MLP	570.05 $\pm$ 0.98

At $C = 19$, Temp-MLP standalone achieves the best performance (568.68 min), improving by approximately 6 minutes with notably reduced variance. Option A and Option C individually outperform the baseline by almost 4 minutes. However, combining components generally does not yield additional gains.

Scaling to $C = 49$. Table 4.6 presents results at the larger graph size with 100 epochs.

Table 4.6: Decoder MLP variant comparison ($C = 49$, 100 epochs, batch size = 256).

Configuration	Mean $\pm$ SD (min)	Δ vs. Baseline
Baseline	1095.93 $\pm$ 73.34	–
Temp-MLP only	1041.57 $\pm$ 19.61	–4.96%
Step-MLP + Temp-MLP	1049.43 $\pm$ 22.91	–4.24%
Option C only	1061.66 $\pm$ 37.07	–3.13%
Step-MLP + Option C + Temp-MLP [†]	1062.48 $\pm$ 25.23	–3.05%
Step-MLP (v2_light)	1068.77 $\pm$ 22.56	–2.48%
Option A only	1158.85 $\pm$ 67.32	+5.74%

[†]Evaluation batch size reduced to 256 (default: 1024) due to CUDA memory constraints.

At $C = 49$, Temp-MLP alone yields the strongest performance (1041.57 min), delivering a 54-minute (4.96%) improvement over the baseline. Option A performs worse than the baseline, suggesting that post-attention glimpse refinement is not beneficial and that it is better to retain the linear transformation rather than a non-linear one. Combining components does not provide meaningful gains over the best standalone variant.

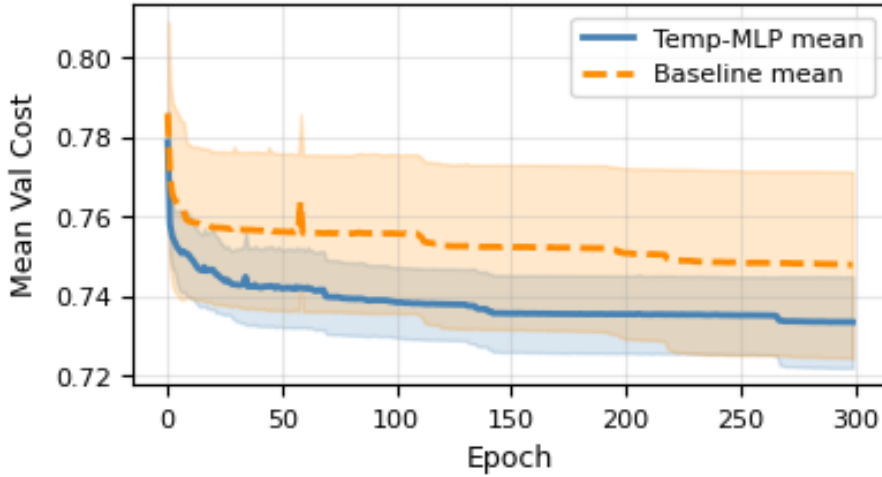


Figure 4.2: Mean validation cost $\pm$ standard deviation across three seeds (1234, 5678, 9012) over 300 epochs at $C = 49$ for Temp-MLP (solid blue) and Baseline (dashed orange). The shaded regions represent $\pm$ one standard deviation. The Temp-MLP mean remains consistently below the baseline throughout training, with a stable gap from approximately epoch 30 onwards.

Due to computational and time limitations, the experiments in the main

study were restricted to 100 epochs. However, it is still worth investigating the impact of longer training. Therefore, both the baseline and Temp-MLP were trained for 300 epochs across all three seeds, which corresponds to triple the default compute budget used in the main experiments. Figure 4.2 shows the mean validation cost across the three seeds over the full 300 epochs.

The mean curve confirms that Temp-MLP maintains a consistent advantage over the baseline throughout the training process. Around epochs 20–30, the gap begins to open and remains almost stable for the rest of training. Importantly, Temp-MLP shows comparatively lower variance across seeds than the baseline, indicating more consistent training behaviour across different random initialisations.

Neither model is fully converged at epoch 300, as both curves are still slowly declining. However, the rate of improvement clearly diminishes in the later epochs, suggesting that additional training would likely provide only marginal gains. The relative ordering between the two models is also unlikely to change.

Summary and Recommendation.

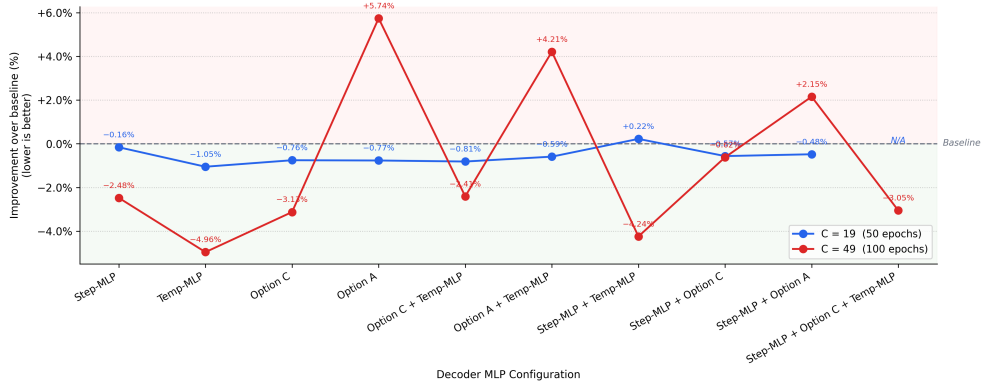


Figure 4.3: Summary of decoder MLP variant performance at $C = 19$ and $C = 49$.

The Temp-MLP produces the most significant and consistent improvement at both graph sizes, with notably low variance. At $C = 49$, it achieves a 4.96% cost reduction. Option A (post-attention MLP) degrades performance at larger scales, suggesting that post-attention glimpse refinement is not beneficial and that it is better to retain the linear transformation rather than a non-linear one. Since the individual components outperform their combinations, Temp-MLP standalone is chosen as the best configuration for the final model comparison (Section 4.10). As discussed earlier, due to computational and time limitations, the experiments in the main study

were restricted to 100 epochs, where a single 100-epoch run at $C = 49$ takes approximately 360–650 minutes per seed. While 300-epoch runs confirmed that the Temp-MLP advantage is stable and persistent, running all configurations for 300 epochs was not feasible within the available compute budget. The 100-epoch results are therefore used consistently across all experiments to ensure fair and comparable evaluation. As shown in Figure 4.2, the relative ordering between Temp-MLP and the baseline is stable from epoch 30 onwards, providing confidence that the conclusions drawn from 100-epoch experiments are reliable despite the limited training budget.

4.4 Batch Size and Training Stability

During initial experiments at $C = 49$, CUDA memory errors were encountered, and as a solution the batch size was reduced (original size was 512). This reduction introduced training instability, so batch sizes of 32 and 256 were compared (Table 4.7).

Table 4.7: Batch size effect on test cost and training stability ($C = 49$, 100 epochs).

Model	Batch	Mean $\pm$ SD (min)	Best Epoch
Baseline	32	1086.67 $\pm$ 13.84	9
Baseline	256	1095.93 $\pm$ 73.34	86
Step-MLP v2_light	32	1072.74 $\pm$ 43.31	35
Step-MLP v2_light	256	1068.77 $\pm$ 22.56	97

Batch size 32 produces early best epochs (epochs 9 and 35), indicating training instability. In contrast, batch size 256 converges more stably, with the best epoch found at epochs 86 to 97. Therefore, **batch size 256** is used for all remaining experiments.

4.5 Hyperparameter Optimisation

After selecting the best-performing Step-MLP feature configuration (v2_light with linear time, Section 4.3), the next step is to identify the best hyperparameter combination for training. All experiments in this section use the Step-MLP v2_light configuration, $C = 20$, batch size 256, and 50 epochs.

Three hyperparameters are tuned sequentially instead of computationally expensive grid search: Step-MLP hidden dimension, learning rate, and learning

rate decay. Figure 4.4 shows the results of the hyperparameter sweep.

For hidden dimension, 64 yields the best performance (591.99 minutes) compared to 32 (600.49 minutes) and 128 (603.06 minutes).

Using the selected hidden dimension of 64, learning rate 0.0001 achieves the best result (591.99 minutes), while other values result in performance degradation of 4 to 12 minutes.

The learning rate decay is applied multiplicatively to the learning rate at the end of each epoch. A decay of 0.995 provides a slight (approximately one minute) advantage compared to the alternatives.

The finalised hyperparameter combination is `step_mlp_dim = 64`, `learning_rate = 0.0001`, and `lr_decay = 0.995`. Table 4.8 summarises the selected values.

Table 4.8: Selected hyperparameters ($C = 20$, Step-MLP v2_light, batch size 256).

Hyperparameter	Values Tested	Selected	Mean $\pm$ SD (min)
Step-MLP dim	32, 64, 128	64	591.99 $\pm$ 3.02
Learning rate	5e-5, 1e-4, 1.5e-4, 2e-4	1e-4	591.99 $\pm$ 3.02
LR decay	0.99, 0.995, 0.998	0.995	591.99 $\pm$ 3.02

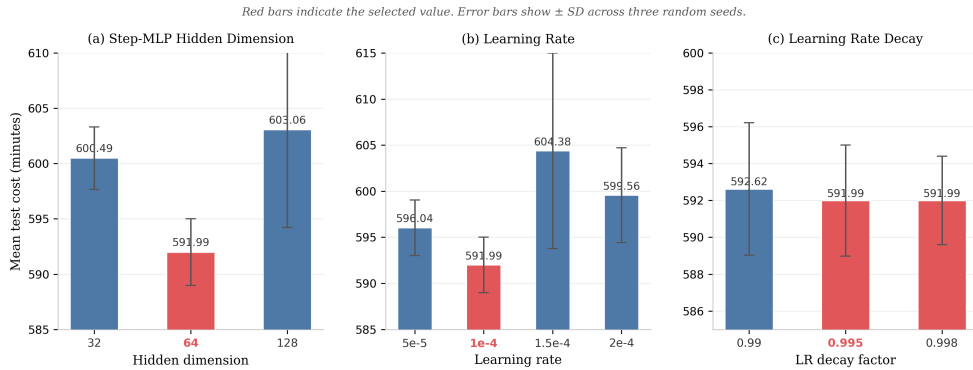


Figure 4.4: Hyperparameter sweep results: Step-MLP hidden dimension (left), learning rate (centre), and learning rate decay (right). All experiments use $C = 20$, batch size 256, 50 epochs.

4.6 Cost-Aware Gating

This section investigates the effect of incorporating cost-aware gating into the M1 model, which adds a heuristic bias to the decoder’s logits (Section 3.5).

Three heuristic functions are evaluated: Nearest Neighbour (NN), Nearest Neighbour Random (NNR), and one-step lookahead (NN+1). The reference baseline for $C = 19$ is 565.50 minutes and for $C = 49$ is 1095.93 minutes (Table 4.2).

Fixed Lambda ($C = 19$). Figure 4.5 shows the effect of various λ values for the NN heuristic at $C = 19$. The optimal range is $\lambda \in [0.25, 0.75]$, and all values within this range outperform the baseline. When λ exceeds 1.0, performance degrades substantially, indicating that over-weighting the heuristic suppresses the learned attention signal.

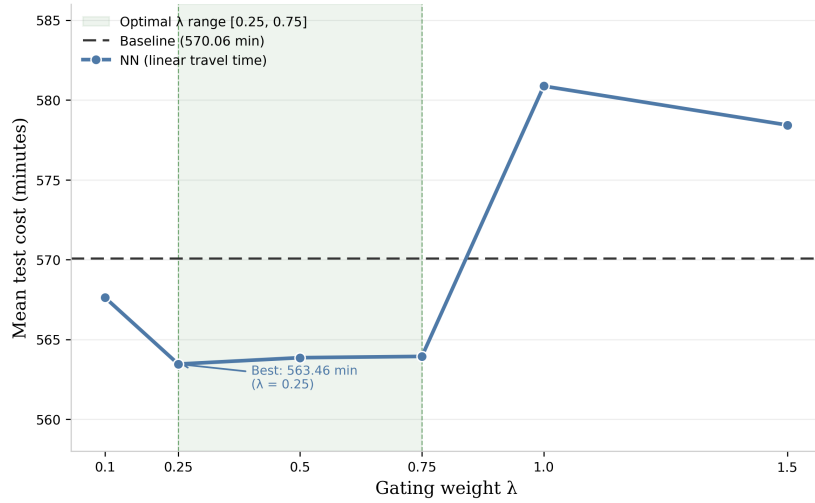


Figure 4.5: Fixed lambda sensitivity with NN heuristic ($C = 19$, seed 1234).

Table 4.9 compares the three heuristic functions with the previously identified fixed λ value. Ideally, the best fixed λ value would be identified separately for each heuristic. However, due to computational limitations, the NN best fixed λ value is applied to the other two methods as well. The NN heuristic achieves the best performance among the three, yet is unable to beat the baseline at this graph size.

Table 4.9: Fixed lambda results by heuristic ($C = 19$, 100 epochs).

Metric	Baseline	NN	NNR	NN+1
Mean $\pm$ SD (min)	565.50 $\pm$ 0.58	566.80 $\pm$ 0.58	572.90 $\pm$ 3.70	570.21 $\pm$ 3.24

Learnable λ was also tested using gradient descent but underperformed fixed λ across all heuristics (575.43 min for NN vs. 566.80 min with fixed

λ). Nonlinear transformations (exponential and piecewise) were also tested but failed to outperform the linear approach (575.43 min and 567.83 min, respectively, vs. baseline 565.50 min).

Scaling to $C = 49$. Since the cost-aware gating mechanism failed to beat the baseline at $C = 19$, it is worth investigating how it performs at $C = 49$. The best-performing configuration at $C = 19$ (NN heuristic, fixed $\lambda = 0.25$) was used to evaluate $C = 49$ (Table 4.10).

Table 4.10: Cost-aware gating results ($C = 49$, 100 epochs, batch size 256, fixed $\lambda = 0.25$).

Configuration	Mean $\pm$ SD (min)	Δ vs. Baseline
Baseline	1095.93 $\pm$ 73.34	—
Cost-aware only (NN)	1055.75 $\pm$ 25.14	−3.67%
Option C + Cost-aware (NN)	1068.17 $\pm$ 26.05	−2.53%
Temp-MLP + Cost-aware (NN)	1062.78 $\pm$ 35.90	−3.02%

Unlike at $C = 19$, cost-aware gating leads to a significant improvement at $C = 49$. Cost-aware gating with the NN heuristic alone produces an improvement of approximately 40 minutes (3.67%) compared to the baseline. Combining with other decoder components does not provide additional gains, so standalone cost-aware gating yields the best performance. This suggests that cost-aware gating is particularly beneficial at larger problem sizes, as the model struggles with larger problems and cost-aware gating provides additional guidance through cost information to produce better results.

Summary and Recommendation. At $C = 19$, cost-aware gating does not outperform the baseline. Learnable λ underperformed fixed λ , and nonlinear transformations failed to improve upon the linear approach.

At $C = 49$, cost-aware gating produces a notable improvement (3.67%) compared to the baseline. Combining with other decoder components does not provide additional gains.

The best cost-aware configuration is: fixed $\lambda = 0.25$, NN heuristic, linear transformation.

4.7 Time Slicing with Safe Refresh

The baseline M1 encoder feeds the full 24-hour traffic data. Time slicing replaces this with a forward window of W time bins starting from a given start bin, thereby reducing the input dimensionality. When the decoder’s

current time approaches the end of the window, a safe refresh mechanism re-encodes (Section 3.5).

Before presenting the experimental results, the motivating hypotheses are introduced. These hypotheses were formulated based on the structure of the TDTSP and the architecture of the baseline model (Section 3.5):

- **H1:** Shorter time windows improve solution quality by reducing noise.
- **H2:** More frequent encoder refresh improves performance.
- **H3:** Optimal configurations depend on graph size.
- **H4:** Start time has a meaningful impact on solution quality.
- **H5:** Time slicing reduces computational cost with small accuracy loss.

These hypotheses are assessed in the summary and recommendations.

The experiments evaluate three configuration choices: window size (W), refresh strategy, and start time. Results are presented across graph sizes $C = 19, 20, 29$, and 49 . The baseline references are 574.73 minutes ($C = 19$, 50 epochs), 766.79 minutes ($C = 29$), and 1095.93 minutes ($C = 49$).

Window Size Comparison. Four window sizes ($W = 3, 6, 9$, and forward window) were evaluated across three graph sizes with fixed start bin $= 3$ and one-time refresh. Figure 4.6 summarises the results.

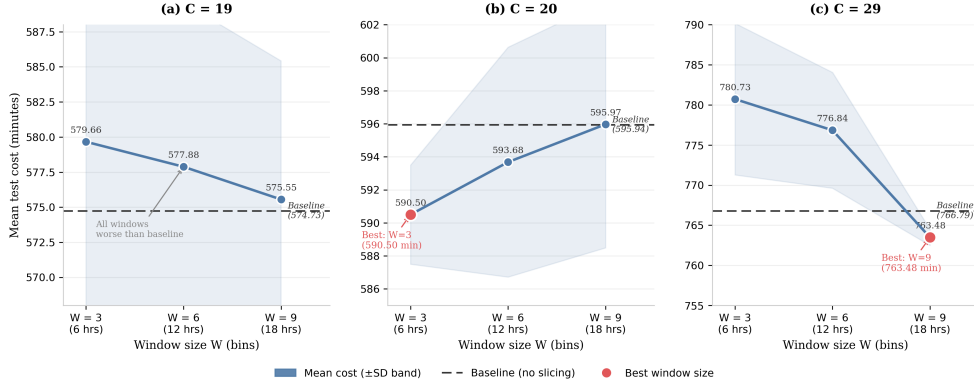


Figure 4.6: Mean test cost vs. window size W for $C = 19$, $C = 20$, and $C = 29$ (start bin $= 3$, one-time refresh, 50 epochs).

At $C = 19$, no window improves upon the baseline, though $W = 9$ performs best among the windows tested. At $C = 20$, almost all time slicing configurations achieve performance at or better than the baseline. Among

them, $W = 3$ achieves the best mean cost (590.50 min) with the lowest variance. To continue investigating the effect of time windows at larger problem sizes, consider $C = 29$. At $C = 29$, $W = 9$ yields better performance than the baseline, as narrower windows (3 and 6) appear insufficient for larger tours. The optimal window size thus depends on graph size.

Refresh Strategy Comparison. Four refresh strategies were evaluated: one-time, periodic (intervals 0.25 and 0.50), k-moves lookahead buffer, and combined. Table 4.11 summarises the best-performing strategy for each graph size.

Table 4.11: Best refresh strategy per graph size.

Graph Size	Window	Best Strategy	Mean $\pm$ SD (min)	Δ vs. Baseline
$C = 19$	$W = 9$	Periodic (0.25)	572.55 ± 2.86	-0.38%
$C = 20$	$W = 3$	One-time	590.50 ± 3.08	-0.91%
$C = 29$	$W = 9$	One-time	763.48 ± 1.11	-0.43%

All of the best refreshing strategies for graph sizes 19, 20, and 29 beat the baseline. At $C = 19$ and 20, periodic refresh (interval = 0.25) performs best. At $C = 29$, periodic refresh (0.50) just beats the baseline, but one-time performed best. One-time refresh is selected for future analysis due to its simplicity and competitive performance with low variance.

Start Time Sensitivity. Start bins 0, 3, 6, and 9 were evaluated for each graph size. Start bin 3 consistently performs best across all graph sizes. Start bin 0 provides insufficient context, while bins 6 and 9 may introduce irrelevant next-day data when combined with larger windows.

Validation at $C = 49$. Using the best configuration ($W = 3$, start bin = 3), time slicing is validated at $C = 49$ with 100 epochs (Table 4.12).

Table 4.12: Time slicing validation ($C = 49$, $W = 3$, start bin = 3, 100 epochs).

Strategy	Mean $\pm$ SD (min)	Δ vs. Baseline
Baseline	1095.93 ± 73.34	—
One-time	1094.39 ± 43.17	-0.14%
Periodic (0.25)	1082.81 ± 26.19	-1.20%

At $C = 49$, periodic (0.25) achieves the best performance of 1082.81 minutes, an improvement of 13 minutes (1.20%) over the baseline. More frequent encoder updates contribute to better solution quality at larger graph sizes.

Computational Efficiency. Table 4.13 compares training time and solution quality.

Table 4.13: Computational efficiency: time slicing vs. baseline.

Graph Size	Best Config	Δ Cost	Time Overhead
$C = 19$	$W = 9$, Periodic (0.25)	-0.38%	+2.49%
$C = 20$	$W = 3$, one-time	-0.91%	+1.70%
$C = 29$	$W = 9$, one-time	-0.43%	+8.33%
$C = 49$	$W = 3$, Periodic (0.25)	-1.19%	+10.72%

Time slicing improves solution quality through regular encoder refreshing and the exclusion of noisy, irrelevant data. However, it adds training overhead due to the refresh mechanism checking window boundaries and re-running the encoder. Rather than saving computation, time slicing trades additional training time for better solutions.

Summary and Recommendation. Table 4.14 consolidates the best time-slicing configuration and its outcome at each graph size. The five hypotheses defined at the outset are assessed below.

Table 4.14: Time slicing summary - best configuration per graph size.

Graph Size	Best Config	Mean Cost (min)	Δ vs. Baseline	Time Overhead
$C = 19$	$W = 9$, start=3, Periodic (0.25)	572.55	-0.38%	+2.49%
$C = 20$	$W = 3$, start=3, one-time	590.50	-0.91%	+1.70%
$C = 29$	$W = 9$, start=3, one-time	763.48	-0.43%	+3.8%
$C = 49$	$W = 3$, start=3, Periodic (0.25)	1082.81	-1.19%	+10.72%

The five hypotheses defined at the outset of this section are now assessed in light of the experimental findings.

H1 (Shorter windows improve quality) - Mostly rejected. The forward window ($W = 9$) outperforms narrow windows in 2/3 graph sizes ($C = 19$ and $C = 29$). This suggests that feeding the encoder with current and all remaining future data is more beneficial.

H2 (Frequent refresh improves performance) - Mostly confirmed. One-time refresh was selected as the practical choice because of its simplicity, but periodic refresh wins in 3/4 cases (except $C = 29$).

H3 (Optimal config depends on graph size) - Confirmed. Window sizes, refresh strategies, and start bins vary with graph size, confirming that time slicing must be tuned per problem scale.

H4 (Start time impacts quality) - Confirmed with caution. Start bin 3 performs best, though this may be influenced by experimental setup. To

confirm the truly optimal configuration, a full grid search over all parameters would be required. Nevertheless, the results clearly indicate that the start time bin has a noticeable impact on performance.

H5 (Time slicing reduces cost) - Rejected. Time slicing adds overhead but improves solution quality.

As part of the experimentation process, the current best decoder enhancement (Temp-MLP) was combined with the best-performing time slicing configuration to investigate whether the combination leads to further improvements. The best time slicing configuration at $C = 49$ ($W = 3$, start bin = 3, periodic refresh interval = 0.25) was combined with Temp-MLP and trained for 100 epochs across three seeds. Table 4.15 presents the results.

Table 4.15: Temp-MLP and Time Slicing combination results ($C = 49$, 100 epochs, batch size 256).

Configuration	Mean $\pm$ SD (min)	Δ vs Baseline
Baseline	1095.93 $\pm$ 73.34	–
Temp-MLP standalone	1041.57 $\pm$ 19.61	–4.96%
Time Slicing only ($W = 3$, start=3, Periodic 0.25)	1082.81 $\pm$ 26.19	–1.19%
Temp-MLP + Time Slicing	1063.75 $\pm$ 6.47	–2.94%

The combination achieves a 2.94% improvement over the baseline, confirming that both extensions individually contribute meaningful gains. However, the combined configuration does not outperform Temp-MLP standalone (1063.75 min vs 1041.57 min). This is consistent with the pattern observed across other combinations throughout this chapter, where individual components consistently outperform their combinations. Notably, the combination exhibits the lowest variance of any configuration tested (± 6.47 min). This suggests that combining the encoder-level time slicing modification with the decoder-level Temp-MLP produces more stable, but not necessarily better, solutions. The increased stability may be attributed to the complementary nature of the two modifications. Time slicing reduces encoder noise, while Temp-MLP regularises the decoder’s exploration. Together, they constrain the solution space more tightly across different random initialisations.

In summary, time slicing is most valuable as a solution quality enhancer at larger problem sizes, with gains of up to 1.19% at $C = 49$. The forward window ($W = 9$ from start bin 3) is the most broadly effective configuration. Periodic refresh is generally preferable, though one-time refresh remains a competitive and practical alternative.

4.8 Graph-Size-Independent Architecture

The baseline M1 model has a significant practical limitation in that it was originally designed for fixed customer set sizes. To address this, an adaptive average pooling layer is introduced before each projection, mapping any input of arbitrary length to a fixed 128-dimensional representation (Section 3.7).

This section evaluates the pooling fix across two dimensions: (1) within-size performance, and (2) cross-size generalisation.

Within-Size Performance. Table 4.16 compares the baseline and pooling models at $C = 19$ and $C = 29$.

Table 4.16: Within-size and cross-size performance (100 epochs, batch size 256).

Model	Train	Test	Mean $\pm$ SD (min)	Δ vs. Baseline
<i>Within-size</i>				
Baseline	19	19	565.50 $\pm$ 0.58	–
Pooling	19	19	570.51 $\pm$ 2.99	+0.89%
Baseline	29	29	766.79 $\pm$ 7.39	–
Pooling	29	29	768.12 $\pm$ 7.87	+0.17%
<i>Cross-size</i>				
Pooling	19	29	770.15 $\pm$ 10.54	+0.44%
Pooling	29	19	572.52 $\pm$ 1.57	+1.24%

At $C = 19$, the pooling model underperforms by 5.01 minutes (0.89%). At $C = 29$, the difference is only 1.33 minutes (0.17%). Overall, the adaptive pooling modification introduces only a minimal performance degradation of less than 0.89%.

Cross-Size Generalisation. The key innovation of the pooling is that the model can be trained on one graph size and evaluated directly on a different graph size without retraining. Results are included in Table 4.16 above.

Training on $C = 19$ and evaluating on $C = 29$ yields 770.15 minutes (0.44% degradation vs. baseline). Training on $C = 29$ and evaluating on $C = 19$ yields 572.52 minutes (1.24% degradation).

Summary. The adaptive pooling modification enables graph-size independence. A single trained instance of the modified M1 model can be applied to any graph size without retraining, with a solution quality degradation of less than 1%.

4.9 Beam Search

All experiments in previous sections use greedy decoding, in which the decoder selects the highest-probability node at each step. Beam search addresses this limitation by maintaining B candidate partial tours at each step, retaining only the B most promising sequences. At the end of decoding, the best complete tour among the B candidates is returned.

Beam width $B = 3$ is evaluated (Table 4.17).

Table 4.17: Beam search results – solution quality ($B = 3$).

Decoding	Graph Size	Mean $\pm$ SD (min)	Δ vs. Greedy
Greedy	$C = 19$	574.73 ± 10.78	—
Beam $B = 3$	$C = 19$	569.59 ± 1.03	-0.89%
Greedy	$C = 49$	1095.93 ± 73.34	—
Beam $B = 3$	$C = 49$	1095.49 ± 73.11	-0.04%

At $C = 19$, beam search improves performance by 5.14 minutes (0.89%) over greedy decoding. The cross-seed variance also drops sharply (SD = 1.03 vs. 10.78). This suggests that, at this scale, beam search (with width $B = 3$) yields a meaningful improvement in solution quality.

At $C = 49$, the improvement is negligible, at only 0.44 minutes (0.04%). This suggests that the search space at this scale is too large for a narrow beam to explore meaningfully. A computational efficiency comparison between greedy and beam search decoding is left for future work due to instrumentation constraints.

4.10 Comparison with Heuristic Baselines

This section compares the best-performing models from each above section with five heuristics: Nearest Neighbour (NN), Greedy Edge (GE), Nearest Neighbour Random (NNR), NN+1 lookahead, and 2-opt local search. All heuristic models are evaluated on the same 1,000 test instances used in the training process (each epoch’s validation process at the end). The same traffic data with the same cubic spline interpolation method is also used. All results are reported as mean $\pm$ standard deviation across three random seeds (1234, 5678, 9012) for neural models, ensuring robust and reliable comparisons.

$C = 19$. Table 4.18 presents the results at $C = 19$.

Table 4.18: Solution quality and inference time comparison at $C = 19$ (mean $\pm$ SD across three seeds).

Model / Heuristic	Mean $\pm$ SD (min)	Time per Instance
<i>Neural Models</i>		
Baseline (M1)	574.73 ± 10.78	0.24 ms
Temp-MLP	568.68 ± 2.12	0.36 ms
Time Slicing ($W = 9$, start=3)	572.55 ± 2.86	0.25 ms
<i>Classical Heuristics</i>		
2-opt	575.65 ± 89.46	7,351.47 ms
Nearest Neighbour (NN)	630.54 ± 105.49	9.80 ms
Greedy Edge	633.71 ± 105.58	10.58 ms
NNR	636.38 ± 101.28	94.58 ms
NN+1	648.86 ± 104.86	110.21 ms

Neural model results are averaged across three random seeds (1234, 5678, 9012). Heuristic baselines are deterministic and evaluated once on the same fixed test set. Cost-Aware Gating is excluded from this table as it did not outperform the baseline at $C = 19$ (see Section 4.6 for full results).

All neural models surpass the classical heuristics at $C = 19$. The baseline M1 outperforms 2-opt by 1 minute while running approximately 30,000 times faster (0.24 ms vs 7,351.47 ms). Temp-MLP achieves the best neural performance (568.68 ± 2.12 min), outperforming 2-opt by approximately 7 minutes with similarly fast inference. The remaining heuristics (NN, GE, NNR, NN+1) all produce performance more than 55 minutes worse compared to the neural models. Cost-Aware Gating is excluded from this table as it did not outperform the baseline at $C = 19$ (see Section 4.6 for full results).

$C = 49$. Table 4.19 presents the results at $C = 49$.

Table 4.19: Solution quality and inference time comparison at $C = 49$ (mean $\pm$ SD across three seeds).

Model / Heuristic	Mean $\pm$ SD (min)	Time per Instance
<i>Neural Models</i>		
Baseline (M1)	1095.93 $\pm$ 73.34	0.64 ms
Temp-MLP	1041.57 $\pm$ 19.61	0.94 ms
Cost-Aware (NN, $\lambda = 0.25$)	1055.75 $\pm$ 25.14	6.05 ms
Time Slicing ($W = 3$, start=3)	1082.81 $\pm$ 26.19	0.65 ms
<i>Classical Heuristics</i>		
2-opt	1117.61 $\pm$ 91.79	91,349.73 ms
Greedy Edge	1154.21 $\pm$ 86.93	27.59 ms
Nearest Neighbour (NN)	1168.03 $\pm$ 100.46	23.83 ms
NN+1	1201.84 $\pm$ 96.65	630.93 ms
NNR	1211.52 $\pm$ 93.24	250.65 ms

Neural model results are averaged across three random seeds (1234, 5678, 9012). Heuristic baselines are deterministic and evaluated once on the same fixed test set.

At $C = 49$, the neural models show a clear advantage over all heuristic models. Temp-MLP achieves the best performance (1041.57 $\pm$ 19.61 min), outperforming 2-opt by approximately 76 minutes (6.8%) while running around 97,000 times faster (0.94 ms vs 91,349 ms). The baseline M1 (1095.93 $\pm$ 73.34 min) also outperforms all heuristics, beating 2-opt by approximately 22 minutes. All proposed extensions outperform every classical heuristic at this graph size.

Summary. For both graph sizes ($C = 19$ and $C = 49$), the neural models consistently outperform classical heuristics in solution quality while offering significantly faster inference. The advantage grows with problem size, with neural models outperforming 2-opt by 1.21% at $C = 19$ and up to 6.8% at $C = 49$. These results confirm that the proposed deep reinforcement learning approach achieves both accuracy and scalability advantages over classical methods.

4.11 Summary, Discussion, and Recommendation

This chapter evaluated four different architectural modifications to the M1 baseline on the deterministic TDTSP with a fixed customer set. The chapter

includes the key findings, discussion points, and practical recommendations for deployment.

Summary of Results

Step-MLP and Decoder Variants. Among all decoder modifications, the Temperature MLP (Temp-MLP) is the standout performer for both graph sizes. At $C = 19$, it reduces the mean cost by 6 minutes, and at $C = 49$, it achieves a 4.96% improvement. In contrast, the post-attention glimpse refinement (Option A) degrades performance at $C = 49$, suggesting that it is better to retain the original linear projection rather than applying a nonlinear transformation to the glimpse vector. The Step-MLP and Option C produce better results at $C = 49$, but combining them with the Temp-MLP reduces performance. This implies that individual components show better performance compared to combinations.

Cost-Aware Gating. This modification is clearly dependent on the graph size. At $C = 19$, the cost-aware gating mechanism did not outperform the baseline, as none of the configurations tested were able to beat it. However, at $C = 49$, standalone cost-aware gating with fixed $\lambda = 0.25$ and the linear NN heuristic produced a meaningful 3.67% improvement. An additional key finding at $C = 19$ was that fixed lambda outperforms learnable lambda. Combining cost-aware gating with other components degraded performance in all tested combinations.

Time Slicing with Safe Refresh. Time slicing consistently adds training overhead (approximately 2.5% at $C = 19$ and 10.7% at $C = 49$), but improves solution quality at both graph sizes (0.38% and 1.19%, respectively). The optimal time-slicing configurations depend on the graph size. The best configuration is $W = 3$ with one-time refresh at $C = 20$, and $W = 9$ with periodic refresh (interval = 0.25) at $C = 19$ and $C = 49$. In most cases, periodic refresh of the encoder improves results.

Graph-Size-Independent Architecture. Introducing the adaptive pooling modification successfully mitigates the cross-size generalisation limitation with minimal quality degradation (at most 1.24% relative to size-specific baselines).

Comparison with Heuristics. All best-performing neural configurations outperform all classical heuristics in mean solution quality at both $C = 19$ and $C = 49$. The 2-opt model is the most competitive among the heuristic models, but requires a substantially longer inference time compared to the neural models.

Overall Recommendation

Based on the experimental findings, the following practical recommendations are made:

- **For small problem sizes ($C = 19/20$):** The Temp-MLP standalone is the recommended extension. Time slicing with $W = 9$, $\text{start} = 3$, and periodic refresh (interval = 0.25) also shows better performance, but the Temp-MLP standalone is highly recommended at this scale. Cost-aware gating should be avoided at this scale.
- **For larger problem sizes ($C = 49$):** The Temp-MLP remains the best single-component choice. However, standalone cost-aware gating (NN, fixed $\lambda = 0.25$) and time slicing with periodic refresh ($W = 3$, interval = 0.25) also show better performance compared to the baseline and are worth considering.
- **Adaptive pooling** mitigates the graph-size dependency with a quality trade-off of at most 1.24% compared to the size-specific baseline, and is worth considering in future developments.
- **Individual components** may show better performance, but combining them does not guarantee further gains. Therefore, it is advisable to evaluate components individually before finalising a configuration.

Chapter 5

Conclusion

5.1 Conclusion

The baseline model for this thesis was introduced by Zhang et al. (2023), and the thesis focuses primarily on the deterministic Time-Dependent Travelling Salesman Problem (TDTSP) with a fixed customer set. The key innovation is the introduction of three architectural extensions to the M1 model, all of which are evaluated on real-world Beijing traffic data across customer set sizes of 19 to 49 nodes. The three main extensions are the Step-MLP with decoder variants, cost-aware gating, and time slicing with refresh mechanisms, each targeting a specific gap in the baseline. Since the M1 model is dependent on the graph size and is limited to a fixed graph size, retraining from scratch is required whenever a new customer base is introduced.

The **Step-Aware Decoder (Step-MLP) and Temperature MLP (Temp-MLP)** introduce tour-progress signals into the decoder query via a lightweight multi-layer perceptron. The simple v2_light feature combination (step ratio, linear time encoding, depot distance, cumulative tour length, and mean distance to unvisited nodes) shows better performance compared to higher-dimensional feature combinations. The Step-MLP adds a context nudge to the decoder, while the Temp-MLP scales the attention logits. Option A and Option C are two further decoder MLP variants, and among all four variants, the Temperature MLP provides the most effective performance at both graph sizes ($C = 19$ and $C = 49$). It achieves 1.05% and 4.96% improvement over the baseline at $C = 19$ and $C = 49$, respectively. In contrast, the post-attention glimpse refinement (Option A) degrades performance at $C = 49$, suggesting that it is better to retain the original linear projection rather than applying a nonlinear transformation to the glimpse vector. The Step-MLP and Option C produce better results at $C = 49$, but combining

them with the Temp-MLP reduces performance. This implies that individual components show better performance compared to combinations.

The **cost-aware gating mechanism** combines the classical heuristic model’s logit with the learned attention signal. This modification is clearly dependent on the graph size. At $C = 19$, the cost-aware gating mechanism did not outperform the baseline, as none of the configurations tested were able to beat it. However, at $C = 49$, standalone cost-aware gating with fixed $\lambda = 0.25$ and the linear NN heuristic produced a meaningful 3.67% improvement. An additional key finding at $C = 19$ was that fixed lambda outperforms learnable lambda, and that a linear transformation is more suitable than a non-linear transformation at this scale. Combining cost-aware gating with other components degraded performance in all tested combinations.

Time slicing with safe refresh replaces the full-day traffic profile with a forward time window (removing past data) and refreshes the encoder when the decoder’s current time exceeds or closes to the window boundary. Time slicing consistently adds training overhead (approximately 2.5% at $C = 19$ and 10.7% at $C = 49$), but improves solution quality at both graph sizes (0.38% and 1.19%, respectively). Time slicing is therefore best understood as a solution quality enhancer rather than a computational shortcut. The optimal time-slicing configurations depend on the graph size. Additionally, combining time slicing with the best decoder enhancement (Temp-MLP) yielded a 2.94% improvement over the baseline and achieved the lowest variance of any configuration tested (± 6.47 min). However, it did not outperform Temp-MLP standalone, which is consistent with the pattern observed across all other combinations in this thesis.

The **graph-size-independent** architecture introduces the adaptive pooling modification, which successfully mitigates the cross-size generalisation limitation with minimal quality degradation (at most 1.24% relative to size-specific baselines).

Finally, all best-performing neural configurations outperform all classical heuristics in mean solution quality at both $C = 19$ and $C = 49$. The 2-opt model is the most competitive among the heuristic models, but requires a substantially longer inference time compared to the neural models.

5.2 Limitations

This work has several limitations that are worth noting. All experiments were conducted on a single real-world dataset from Beijing, and better generalisation would require experimentation with different datasets. Since the baseline paper (Zhang et al., 2023) experimented with a maximum size of

19, this thesis extends the evaluation to 49. However, it is worth extending this further to larger graph sizes, and with the latest developments to the model, there is potential to do so. The hyperparameter search was conducted sequentially rather than through a full grid search or Bayesian optimisation, which may have left better configurations unexplored. The majority of experiments were conducted with 100 training epochs due to computational constraints, which is significantly fewer than the 1,000 epochs used in the original (Zhang et al., 2023) paper. Extended 300-epoch runs confirmed that the relative ordering between Temp-MLP and the baseline is stable, but both models are likely still undertrained and further gains are expected with a larger compute budget. Heuristic baselines were evaluated on a single random seed with 1,000 instances, while neural model results were averaged across three seeds.

5.3 Future Work

Several directions are proposed to extend this work. The Temperature MLP showed the most consistent improvements across both graph sizes and warrants deeper investigation, particularly regarding its behaviour and scaling factors. Furthermore, the combination of Temp-MLP and Time Slicing demonstrated the lowest variance across seeds of any configuration tested. This suggests that the combination warrants further investigation with longer training to determine whether the stability advantage eventually translates into improved solution quality.

The comparison of the best neural model with the heuristic models clearly demonstrated an advantage as the problem size increases. However, this was tested on only two graph sizes, and it is worth experimenting with much larger graph sizes to further validate this trend.

The time slicing mechanism demonstrated that focusing the encoder on relevant future traffic improves solution quality. Currently, the window size is fixed, meaning the encoder is refreshed with the same time window size at every step. However, the optimal window size is closely related to how much of the tour remains. In the early phase of the tour, a wider time window is needed, while in the later phase, a smaller window is sufficient as fewer customers remain. Introducing an adaptive window strategy that self-adjusts rather than using a fixed window size may therefore improve solution quality further.

From a training perspective, replacing the fixed Beijing dataset with a synthetic traffic generator or other real-world datasets would allow a more generalised version of the model to be developed.

The baseline paper by Zhang et al. (2023) explored both the REINFORCE algorithm with a rollout baseline and an actor-critic training framework. Among these, the REINFORCE algorithm with a rollout baseline produced better results, and therefore the same technique is adopted in this analysis. However, applying the actor-critic framework in combination with the latest modifications introduced in this thesis remains unexplored. Investigating whether actor-critic methods, when paired with the Temp-MLP, cost-aware gating, and time slicing modifications, can yield further improvements represents a promising direction for future work.

Finally, extending the deterministic framework to stochastic travel times would broaden the analysis to more realistic time-dependent routing problems.

Bibliography

- Bello, I., Pham, H., Le, Q. V., Norouzi, M., and Bengio, S. (2017). Neural combinatorial optimization with reinforcement learning. In *International Conference on Learning Representations Workshop*.
- Chen, X. et al. (2025). Dynamic traveling salesman problem with time-dependent stochastic travel times. *Transportation Research Part C: Emerging Technologies*.
- Choo, J., Kwon, Y.-D., Kim, J., Jae, J., Hottung, A., Tierney, K., and Gwon, Y. (2022). Simulation-guided beam search for neural combinatorial optimization. In *Advances in Neural Information Processing Systems*.
- Cordeau, J.-F., Ghiani, G., and Guerriero, E. (2014). Analysis and branch-and-cut algorithm for the time-dependent travelling salesman problem. *Transportation Science*, 48(1):46–58.
- Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research*, 6(6):791–812.
- Dai, H., Khalil, E. B., Zhang, Y., Dilkina, B., and Song, L. (2017). Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems*, volume 30.
- Drakulic, D., Michel, S., Mai, F., Sors, A., and Andreoli, J.-M. (2023). BQ-NCO: Bisimulation quotienting for efficient neural combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36.
- Ehmke, J. F., Campbell, A. M., and Thomas, B. W. (2018). Optimizing for total costs in vehicle routing in urban areas. *Transportation Research Part E: Logistics and Transportation Review*, 116:242–265.
- Gendreau, M., Ghiani, G., and Guerriero, E. (2015). Time-dependent routing problems: A review. *Computers & Operations Research*, 64:189–197.

- Haghani, A. and Jung, S. (2005). A dynamic vehicle routing problem with time-dependent travel times. *Computers & Operations Research*, 32(11):2959–2986.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Spatial pyramid pooling in deep convolutional networks for visual recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(9):1904–1916.
- Ichoua, S., Gendreau, M., and Potvin, J.-Y. (2003). Vehicle dispatching with time-dependent travel times. *European Journal of Operational Research*, 144(2):379–396.
- Joshi, C. K., Laurent, T., and Bresson, X. (2019). An efficient graph convolutional network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227*.
- Kool, W., van Hoof, H., and Welling, M. (2019). Attention, learn to solve routing problems! In *International Conference on Learning Representations*.
- Malandraki, C. and Daskin, M. S. (1992). Time dependent vehicle routing problems: Formulations, properties and heuristic algorithms. *Transportation Science*, 26(3):185–200.
- Malandraki, C. and Dial, R. B. (1996). A restricted dynamic programming heuristic algorithm for the time dependent traveling salesman problem. *European Journal of Operational Research*, 90(1):45–55.
- Mavrovouniotis, M. and Yang, S. (2013). Ant colony optimization with immigrants schemes for the dynamic travelling salesman problem with traffic factors. *Applied Soft Computing*, 13(10):4023–4037.
- Papadimitriou, C. H. (1977). The euclidean travelling salesman problem is NP-complete. *Theoretical Computer Science*, 4(3):237–244.
- Peng, B., Wang, J., and Zhang, Z. (2019). A deep reinforcement learning algorithm using dynamic attention model for vehicle routing problems. In *International Symposium on Intelligence Computation and Applications*.
- Picard, J.-C. and Queyranne, M. (1978). The time-dependent traveling salesman problem and its application to the tardiness problem in one-machine scheduling. *Operations Research*, 26(1):86–110.
- Rosenkrantz, D. J., Stearns, R. E., and Lewis, P. M. (1977). An analysis of several heuristics for the traveling salesman problem. *SIAM Journal on Computing*, 6(3):563–581.

- Schneider, J. (2003). The time-dependent traveling salesman problem. *Future Generation Computer Systems*, 19(1):121–131.
- Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to sequence learning with neural networks. In *Advances in Neural Information Processing Systems*, volume 27.
- Vander Wiel, R. J. and Sahinidis, N. V. (1995). Heuristic bounds and test problem generation for the time-dependent traveling salesman problem. *Transportation Science*, 29(2):167–183.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Vinyals, O., Fortunato, M., and Jaitly, N. (2015). Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28.
- Wu, G., Zhang, Z., Liu, H., and Wang, J. (2021). Solving time-dependent traveling salesman problem with time windows with deep reinforcement learning. In *2021 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 558–563.
- Xin, L., Song, W., Cao, Z., and Zhang, J. (2021a). Multi-decoder attention model with embedding glimpse for solving vehicle routing problems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 12042–12049.
- Xin, L., Song, W., Cao, Z., and Zhang, J. (2021b). Step-wise deep learning models for solving routing problems. *IEEE Transactions on Industrial Informatics*, 17(7):4861–4871.
- Ye, H., Wang, J., Cao, Z., Liang, H., and Li, Y. (2023). DeepACO: Neural-enhanced ant systems for combinatorial optimization. In *Advances in Neural Information Processing Systems*.
- Zhang, Z., Liu, H., Zhou, M., and Wang, J. (2023). Solving dynamic traveling salesman problems with deep reinforcement learning. *IEEE Transactions on Neural Networks and Learning Systems*, 34(4):2119–2132.